
probability
sentences
shows
grammar
model
state
information
speech
set
algorithm
words
language
sentence
models
rules
system
structure
semantic
different
representation
feature
verb
input
parse
since
meaning
rule
first
english
training
based
sequence
parsing
discourse
corpus
number
given
dialogue
lexical
np
noun
observation
relation
expressions
equation
processing
using
hmm
languages
form
recognition
machine
learning
search
vector
simple
problem
text
three
important
next
word
question
sense
case
note
ledge
value
compute
instead
distance
input
verb
feature
human
representation
tagging
features
structure
rules
corpus
language
consider
regular
relations
translation
given
np
lexical
models
sentence
equation
processing
using
hmm
languages
form
different
semantic
rule
first
english
training
grammar
probability
sentences
shows
context
data
whether
frequency
event
function
documents
context-free
string
classifier
verbs
rather
next
word
model
question
probabilistic
sense
discourse
parsing
based
just
three
text
recall
information
state
sequence
compute
value
maximum
output
subject
problem
instead
parse
since
meaning
rule
systems
syntactic
morphological
machine
learning
search
vector
simple
input
verb
feature
algorithm
representation
semantic
structure
different
languages
rules
form
corpus
language
system
hmm
tag
equation
number
test
processing
using
given
dialogue
lexical
np
noun
models
sentence
observation
relation
expressions
part-of-speech
type
terms
relations
translation
regular
consider
semantics
process
represent
analysis
finite-state
natural
unification
class
role
similar
expression
representations
want
recognition
viterbi
distance
phrases
occur
either
start
tree
constraints
ledge
note
case

LLOs

- LLO1: Trình bày được các kiến thức cơ bản về phân tích cú pháp bao gồm bài toán và các tiếp cận giải quyết, các mô hình văn phạm CFG và PCFG, phân tích cú pháp theo tiếp cận thống kê và treebank, đánh giá kết quả phân tích cú pháp
- LLO2: Vận dụng được văn phạm CFG/PCFG trong mô tả cú pháp của ngôn ngữ tự nhiên như đọc hiểu dữ liệu treebank, dựng cây cú pháp thủ công, tính xác suất của cây
- LLO3: Vận dụng được các thuật toán: nhị phân hóa văn phạm phi ngữ cảnh (chuyển văn phạm sang dạng chuẩn Chomsky), lập bảng phân tích sử dụng thuật toán CKY

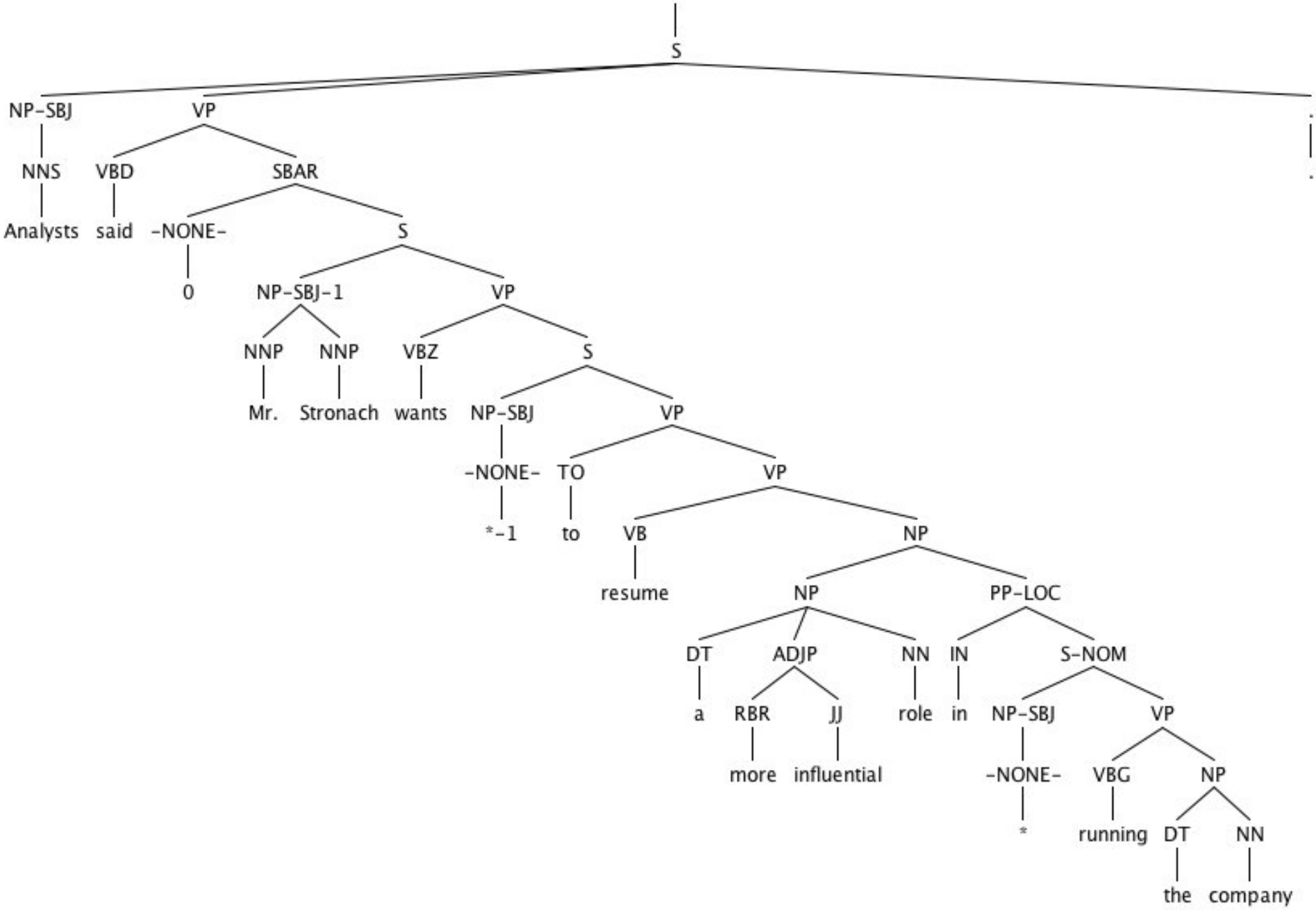


Statistical Natural Language Parsing

Two views of syntactic structure

Two views of linguistic structure: 1. Constituency (phrase structure)

- Phrase structure organizes words into nested constituents.
- How do we know what is a constituent? (Not that linguists don't argue about some cases.)
  - Distribution: a constituent behaves as a unit that can appear in different places:
    - John talked [to the children] [about drugs].
    - John talked [about drugs] [to the children].
    - *John talked drugs to the children about
  - Substitution/expansion/pro-forms:
    - I sat [on the box/right on top of the box/there].
  - Coordination, regular internal structure, no intrusion, fragments, semantics, …


S
NP-SBJ
VP
.
NNS
VBD
SBAR
.
Analysts
said
-NONE-
S
0
NP-SBJ-1
VP
NNP
NNP
VBZ
S
Mr.
Stronach
wants
NP-SBJ
VP
-NONE-
TO
VP
*-1
to
VB
NP
resume
NP
PP-LOC
DT
ADJP
NN
IN
S-NOM
a
RBR
JJ
role
in
NP-SBJ
VP
more
influential
-NONE-
VBG
NP
*
running
DT
NN
the
company

Headed phrase structure

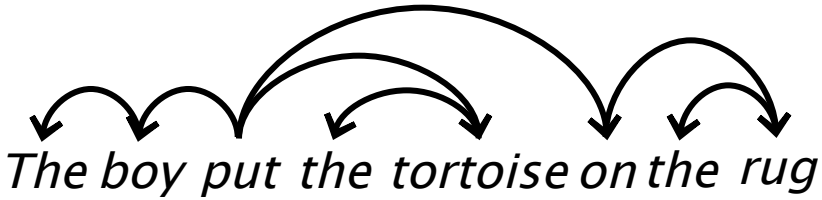
- VP ✹ … VB* …
- NP ✹ … NN* …
- ADJP ✹ … JJ* …
- ADVP ✹ … RB* …

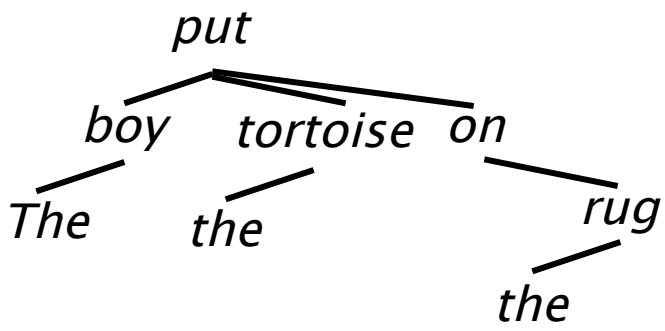
- SBAR(Q) ✹ S|SINV|SQ ✹ … NP VP …

- Plus minor phrase types:
  - QP (quantifier phrase in NP), CONJP (multi word constructions: *as well as*), INTJ (interjections), etc.

Two views of linguistic structure: 2. Dependency structure

- Dependency structure shows which words depend on (modify or are arguments of) which other words.







Statistical Natural Language Parsing

Parsing: The rise of data and statistics

Pre 1990 ("Classical") NLP Parsing

- Wrote symbolic grammar (CFG or often richer) and lexicon

S ✹ NP VP	NN ✹ *interest*
NP ✹ (DT) NN	NNS ✹ *rates*
NP ✹ NN NNS	NNS ✹ *raises*
NP ✹ NNP	VBP ✹ *interest*
VP ✹ V NP	VBZ ✹ *rates*

- Used grammar/proof systems to prove parses from words
- This scaled very badly and didn't give coverage. For sentence:

  Fed raises interest rates 0.5% in effort to control inflation

  - Minimal grammar: 36 parses
  - Simple 10 rule grammar: 592 parses
  - Real-size broad-coverage grammar: millions of parses

Classical NLP Parsing: The problem and its solution

- Categorical constraints can be added to grammars to limit unlikely/weird parses for sentences
  - But the attempt make the grammars not robust
    - In traditional systems, commonly 30% of sentences in even an edited text would have *no* parse.
- A less constrained grammar can parse more sentences
  - But simple sentences end up with ever more parses with no way to choose between them
- We need mechanisms that allow us to find the most likely parse(s) for a sentence
  - Statistical parsing lets us work with very loose grammars that admit millions of parses for sentences but still quickly find the best parse(s)

The rise of annotated data: The Penn Treebank

[Marcus et al. 1993, *Computational Linguistics*]

```
( (S
    (NP-SBJ (DT The) (NN move))
    (VP (VBD followed)
      (NP
        (NP (DT a) (NN round))
        (PP (IN of)
          (NP
            (NP (JJ similar) (NNS increases))
            (PP (IN by)
              (NP (JJ other) (NNS lenders)))
            (PP (IN against)
              (NP (NNP Arizona) (JJ real) (NN estate) (NNS loans))))))
      (, ,)
      (S-ADV
        (NP-SBJ (-NONE- *))
        (VP (VBG reflecting)
          (NP
            (NP (DT a) (VBG continuing) (NN decline))
            (PP-LOC (IN in)
              (NP (DT that) (NN market)))))))
    (. .)))
```

The rise of annotated data

- Starting off, building a treebank seems a lot slower and less useful than building a grammar

- But a treebank gives us many things
  - Reusability of the labor
    - Many parsers, POS taggers, etc.
    - Valuable resource for linguistics
  - Broad coverage
  - Frequencies and distributional information
  - A way to evaluate systems



CFGs and PCFGs

(Probabilistic)
Context-Free
Grammars

A phrase structure grammar

S ✹ NP VP

VP ✹ V NP

VP ✹ V NP PP

NP ✹ NP NP

NP ✹ NP PP

NP ✹ N

NP ✹ *e*

PP ✹ P NP

N ✹ people

N ✹ fish

N ✹ tanks

N ✹ rods

V ✹ people

V ✹ fish

V ✹ tanks

P ✹ with

people fish tanks

people fish with rods

Phrase structure grammars
= context-free grammars (CFGs)

- G = (T, N, S, R)
  - T is a set of terminal symbols
  - N is a set of nonterminal symbols
  - S is a start symbol (S ∈ N)
  - R is a set of rules/productions of the form X ✹ ♑
    - X ∈ N and ♑ ∈ (N ∪ T)*

- A grammar G generates a language L.

Phrase structure grammars in NLP

- G = (T, C, N, S, L, R)
  - T is a set of terminal symbols
  - C is a set of preterminal symbols
  - N is a set of nonterminal symbols
  - S is the start symbol (S ∈ N)
  - L is the lexicon, a set of items of the form X ✹ x
    - X ∈ P and x ∈ T
  - R is the grammar, a set of items of the form X ✹ γ
    - X ∈ N and γ ∈ (N ∪ C)*
- By usual convention, S is the start symbol, but in statistical NLP, we usually have an extra node at the top (ROOT, TOP)
- We usually write *e* for an empty sequence, rather than nothing

A phrase structure grammar

S ✹ NP VP

VP ✹ V NP

VP ✹ V NP PP

NP ✹ NP NP

NP ✹ NP PP

NP ✹ N

NP ✹ *e*

PP ✹ P NP

N ✹ *people*

N ✹ *fish*

N ✹ *tanks*

N ✹ *rods*

V ✹ *people*

V ✹ *fish*

V ✹ *tanks*

P ✹ *with*

people fish tanks

people fish with rods

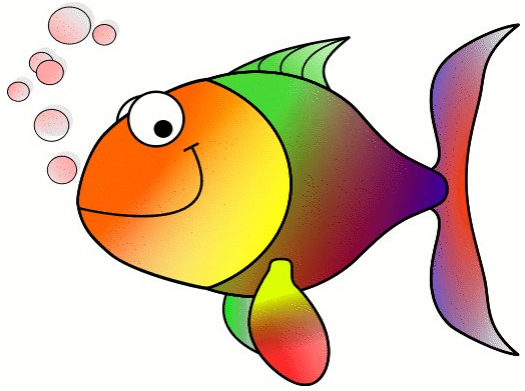
Probabilistic – or stochastic – context-free grammars (PCFGs)

- G = (T, N, S, R, P)
  - T is a set of terminal symbols
  - N is a set of nonterminal symbols
  - S is the start symbol (S ∈ N)
  - R is a set of rules/productions of the form X ✹ ♑
  - P is a probability function
    - P: R ✹ [0,1]
    - $\forall X \in N, \sum_{X \rightarrow \gamma \in R} P(X \rightarrow \gamma) = 1$
- A grammar G generates a language model L.

$$\sum_{\gamma \in T^*} P(\gamma) = 1$$

A PCFG

S ✹ NP VP	1.0
VP ✹ V NP	0.6
VP ✹ V NP PP	0.4
NP ✹ NP NP	0.1
NP ✹ NP PP	0.2
NP ✹ N	0.7
PP ✹ P NP	1.0



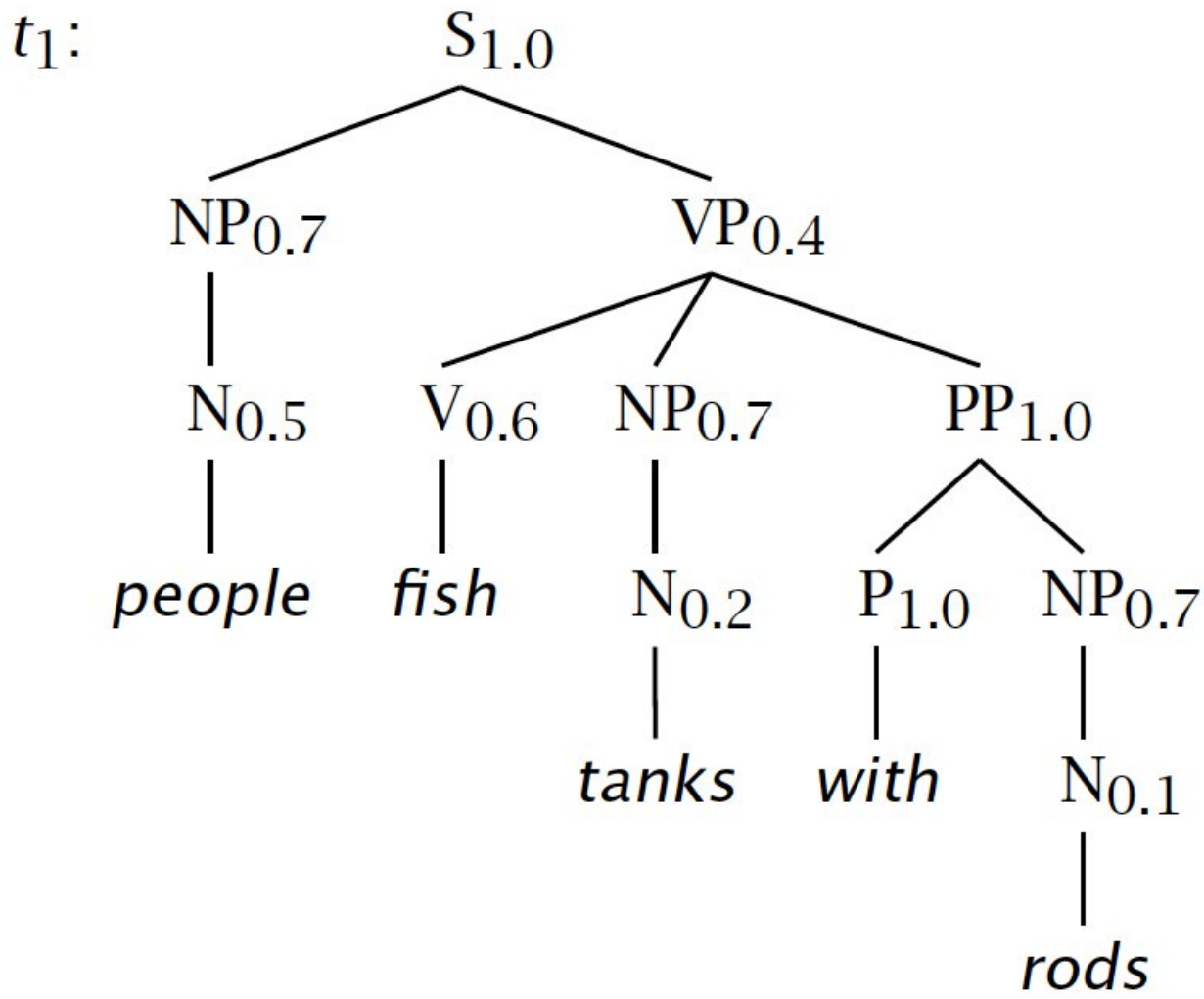
N ✹ *people*	0.5
N ✹ *fish*	0.2
N ✹ *tanks*	0.2
N ✹ *rods*	0.1
V ✹ *people*	0.1
V ✹ *fish*	0.6
V ✹ *tanks*	0.3
P ✹ *with*	1.0

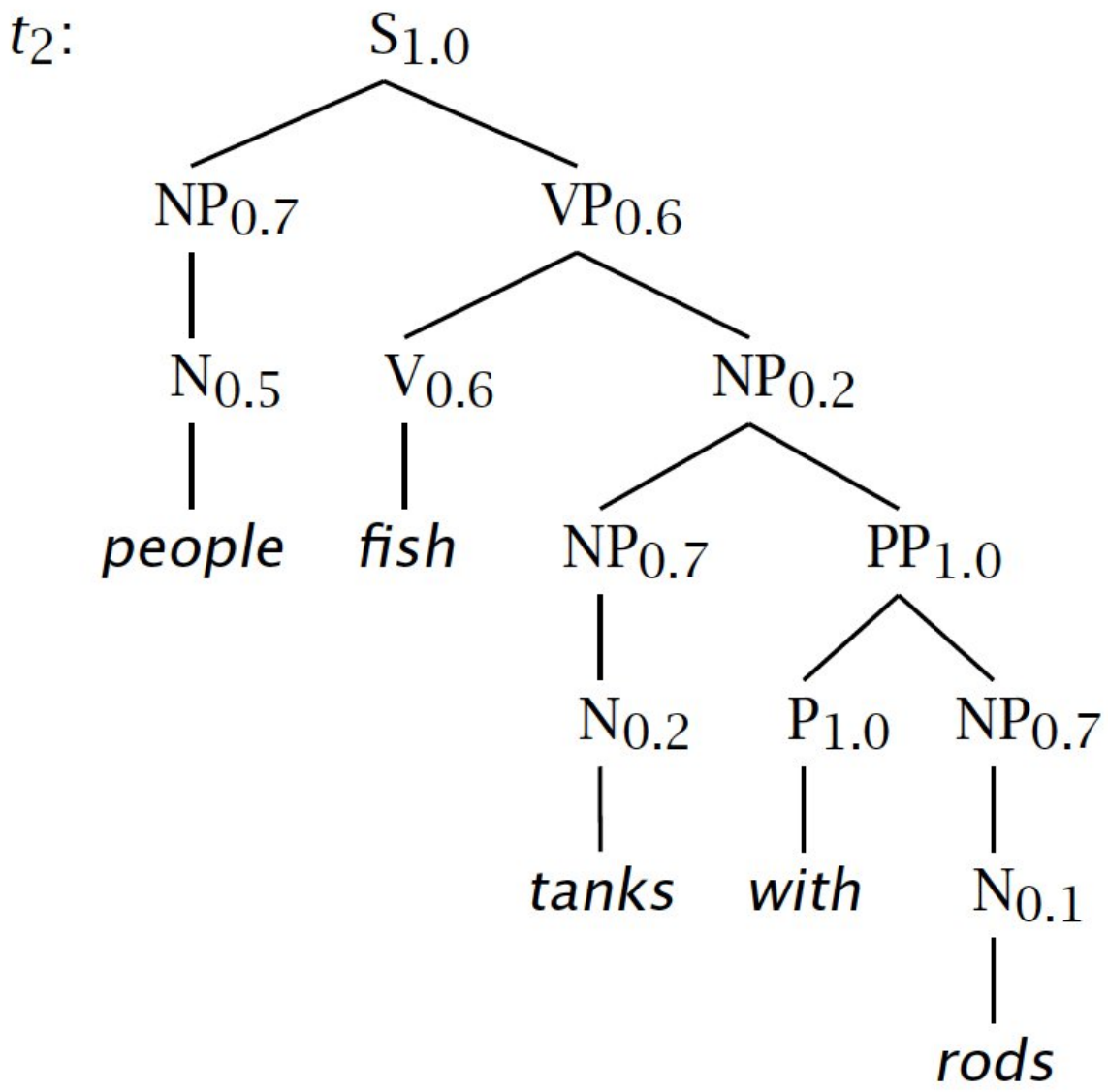
[With empty NP removed
so less ambiguous]

The probability of trees and strings

- P(t) – The probability of a tree t is the product of the probabilities of the rules used to generate it.
- P(s) – The probability of the string s is the sum of the probabilities of the trees which have that string as their yield

$$P(s) = \Sigma_j P(s, t) \text{ where } t \text{ is a parse of } s$$
$$= \Sigma_j P(t)$$


t_1:
$S_{1.0}$
$NP_{0.7}$
$VP_{0.4}$
$N_{0.5}$
$V_{0.6}$
$NP_{0.7}$
$PP_{1.0}$
people
fish
$N_{0.2}$
$P_{1.0}$
$NP_{0.7}$
tanks
with
$N_{0.1}$
rods

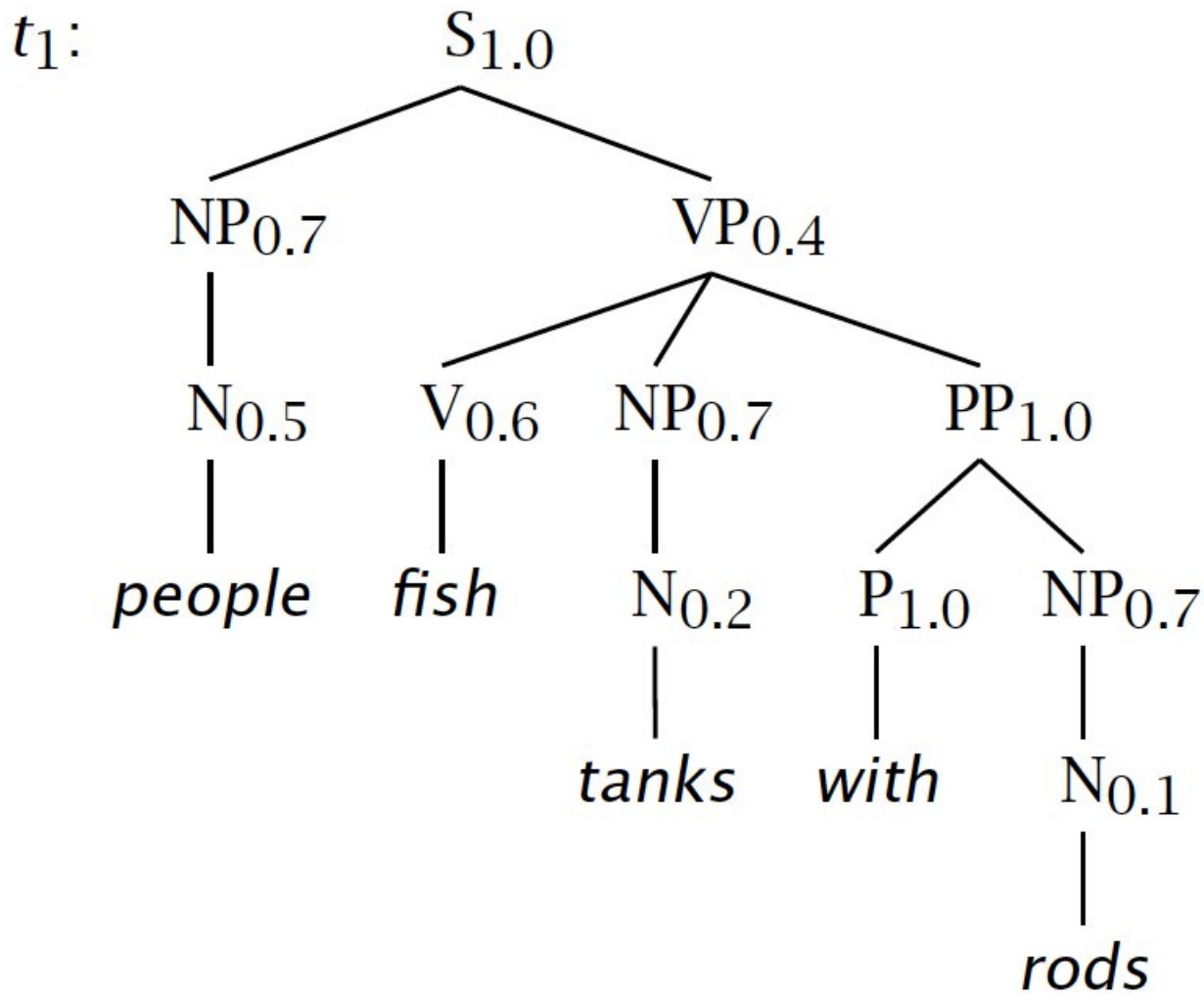

t_2:
$S_{1.0}$
$NP_{0.7}$
$VP_{0.6}$
$N_{0.5}$
$V_{0.6}$
$NP_{0.2}$
people
fish
$NP_{0.7}$
$PP_{1.0}$
$N_{0.2}$
$P_{1.0}$
$NP_{0.7}$
tanks
with
$N_{0.1}$
rods

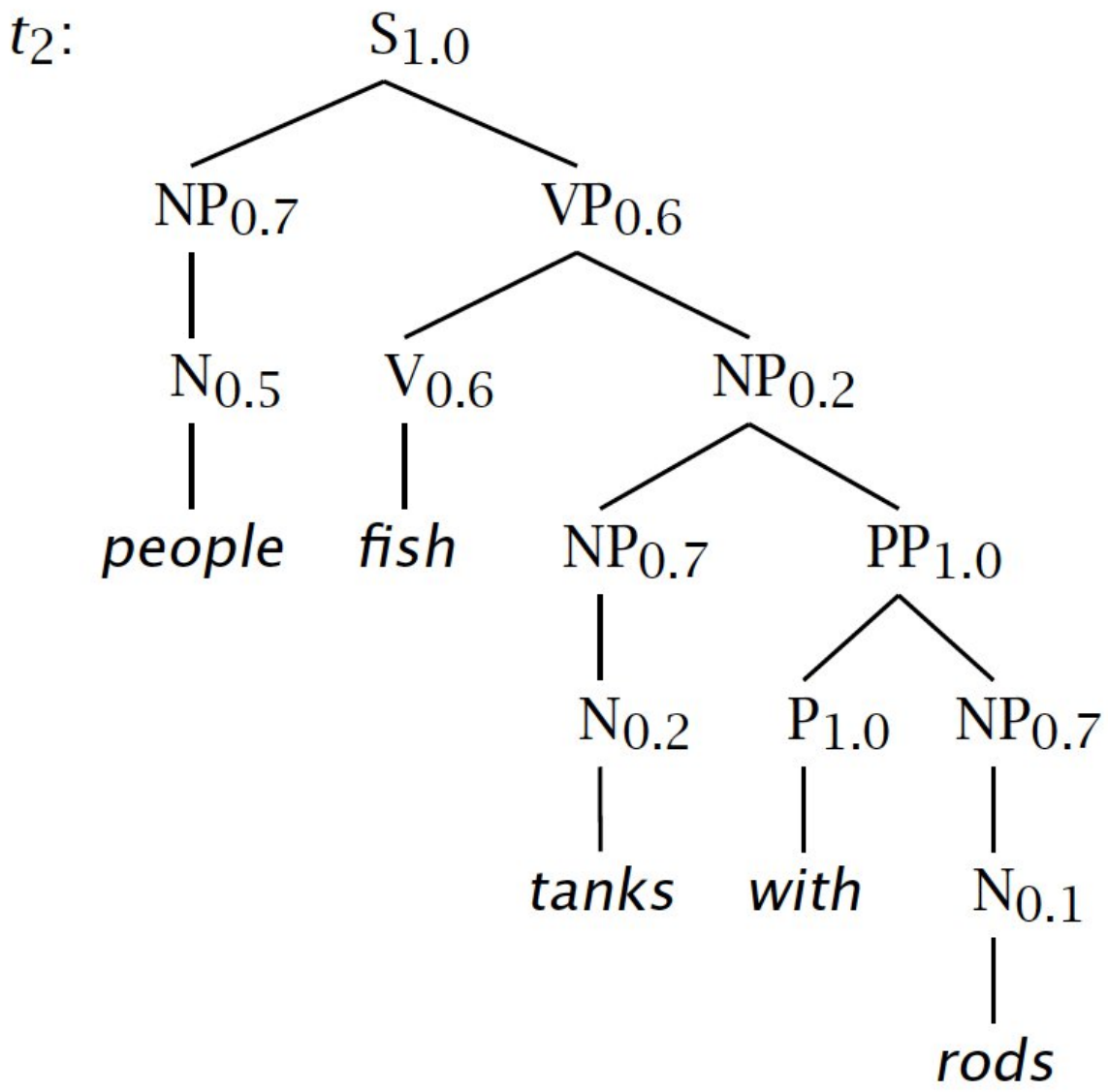
Tree and String Probabilities

- s = *people fish tanks with rods*
- $P(t_1) = 1.0 \times 0.7 \times 0.4 \times 0.5 \times 0.6 \times 0.7 \times 1.0 \times 0.2 \times 1.0 \times 0.7 \times 0.1$
  $= 0.0008232$

  Verb attach
- $P(t_2) = 1.0 \times 0.7 \times 0.6 \times 0.5 \times 0.6 \times 0.2 \times 0.7 \times 1.0 \times 0.2 \times 1.0 \times 0.7 \times 0.1$
  $= 0.00024696$

  Noun attach
- $P(s) = P(t_1) + P(t_2)$
  $= 0.0008232 + 0.00024696$
  $= 0.00107016$


t_1:
$S_{1.0}$
$NP_{0.7}$
$VP_{0.4}$
$N_{0.5}$
$V_{0.6}$
$NP_{0.7}$
$PP_{1.0}$
people
fish
$N_{0.2}$
$P_{1.0}$
$NP_{0.7}$
tanks
with
$N_{0.1}$
rods


t_2:
$S_{1.0}$
$NP_{0.7}$
$VP_{0.6}$
$N_{0.5}$
$V_{0.6}$
$NP_{0.2}$
people
fish
$NP_{0.7}$
$PP_{1.0}$
$N_{0.2}$
$P_{1.0}$
$NP_{0.7}$
tanks
with
$N_{0.1}$
rods



Grammar Transforms

Restricting the grammar form for efficient parsing

Chomsky Normal Form

- All rules are of the form X ✹ Y Z or X ✹ w
  - X, Y, Z ∈ N and w ∈ T
- A transformation to this form doesn't change the weak generative capacity of a CFG
  - That is, it recognizes the same language
    - But maybe with different trees
- Empties and unaries are removed recursively
- n-ary rules are divided by introducing new nonterminals (n > 2)

A phrase structure grammar

S ✹ NP VP

VP ✹ V NP

VP ✹ V NP PP

NP ✹ NP NP

NP ✹ NP PP

NP ✹ N

NP ✹ *e*

PP ✹ P NP

N ✹ *people*

N ✹ *fish*

N ✹ *tanks*

N ✹ *rods*

V ✹ *people*

V ✹ *fish*

V ✹ *tanks*

P ✹ *with*

Chomsky Normal Form steps

S ✹ NP VP

S ✹ VP

VP ✹ V NP

VP ✹ V

VP ✹ V NP PP

VP ✹ V PP

NP ✹ NP NP

NP ✹ NP

NP ✹ NP PP

NP ✹ PP

NP ✹ N

PP ✹ P NP

PP ✹ P

N ✹ *people*

N ✹ *fish*

N ✹ *tanks*

N ✹ *rods*

V ✹ *people*

V ✹ *fish*

V ✹ *tanks*

P ✹ *with*

Chomsky Normal Form steps

S ✹ NP VP

VP ✹ V NP

S ✹ V NP

VP ✹ V

S ✹ V

VP ✹ V NP PP

S ✹ V NP PP

VP ✹ V PP

S ✹ V PP

NP ✹ NP NP

NP ✹ NP

NP ✹ NP PP

NP ✹ PP

NP ✹ N

PP ✹ P NP

PP ✹ P

N ✹ *people*

N ✹ *fish*

N ✹ *tanks*

N ✹ *rods*

V ✹ *people*

V ✹ *fish*

V ✹ *tanks*

P ✹ *with*

Chomsky Normal Form steps

S ✹ NP VP

VP ✹ V NP

S ✹ V NP

VP ✹ V

VP ✹ V NP PP

S ✹ V NP PP

VP ✹ V PP

S ✹ V PP

NP ✹ NP NP

NP ✹ NP

NP ✹ NP PP

NP ✹ PP

NP ✹ N

PP ✹ P NP

PP ✹ P

N ✹ *people*

N ✹ *fish*

N ✹ *tanks*

N ✹ *rods*

V ✹ *people*

S ✹ *people*

V ✹ *fish*

S ✹ *fish*

V ✹ *tanks*

S ✹ *tanks*

P ✹ *with*

Chomsky Normal Form steps

S ✹ NP VP

VP ✹ V NP

S ✹ V NP

VP ✹ V NP PP

S ✹ V NP PP

VP ✹ V PP

S ✹ V PP

NP ✹ NP NP

NP ✹ NP

NP ✹ NP PP

NP ✹ PP

NP ✹ N

PP ✹ P NP

PP ✹ P

N ✹ *people*

N ✹ *fish*

N ✹ *tanks*

N ✹ *rods*

V ✹ *people*

S ✹ *people*

VP ✹ *people*

V ✹ *fish*

S ✹ *fish*

VP ✹ *fish*

V ✹ *tanks*

S ✹ *tanks*

VP ✹ *tanks*

P ✹ *with*

Chomsky Normal Form steps

S ✹ NP VP

VP ✹ V NP

S ✹ V NP

VP ✹ V NP PP

S ✹ V NP PP

VP ✹ V PP

S ✹ V PP

NP ✹ NP NP

NP ✹ NP PP

NP ✹ P NP

PP ✹ P NP

NP ✹ *people*

NP ✹ *fish*

NP ✹ *tanks*

NP ✹ *rods*

V ✹ *people*

S ✹ *people*

VP ✹ *people*

V ✹ *fish*

S ✹ *fish*

VP ✹ *fish*

V ✹ *tanks*

S ✹ *tanks*

VP ✹ *tanks*

P ✹ *with*

PP ✹ *with*

Chomsky Normal Form steps

S ✹ NP VP

VP ✹ V NP

S ✹ V NP

VP ✹ V @VP_V

@VP_V ✹ NP PP

S ✹ V @S_V

@S_V ✹ NP PP

VP ✹ V PP

S ✹ V PP

NP ✹ NP NP

NP ✹ NP PP

NP ✹ P NP

PP ✹ P NP

NP ✹ *people*

NP ✹ *fish*

NP ✹ *tanks*

NP ✹ *rods*

V ✹ *people*

S ✹ *people*

VP ✹ *people*

V ✹ *fish*

S ✹ *fish*

VP ✹ *fish*

V ✹ *tanks*

S ✹ *tanks*

VP ✹ *tanks*

P ✹ *with*

PP ✹ *with*

A phrase structure grammar

S ✹ NP VP

VP ✹ V NP

VP ✹ V NP PP

NP ✹ NP NP

NP ✹ NP PP

NP ✹ N

NP ✹ *e*

PP ✹ P NP

N ✹ *people*

N ✹ *fish*

N ✹ *tanks*

N ✹ *rods*

V ✹ *people*

V ✹ *fish*

V ✹ *tanks*

P ✹ *with*

Chomsky Normal Form steps

S ✹ NP VP

VP ✹ V NP

S ✹ V NP

VP ✹ V @VP_V

@VP_V ✹ NP PP

S ✹ V @S_V

@S_V ✹ NP PP

VP ✹ V PP

S ✹ V PP

NP ✹ NP NP

NP ✹ NP PP

NP ✹ P NP

PP ✹ P NP

NP ✹ *people*

NP ✹ *fish*

NP ✹ *tanks*

NP ✹ *rods*

V ✹ *people*

S ✹ *people*

VP ✹ *people*

V ✹ *fish*

S ✹ *fish*

VP ✹ *fish*

V ✹ *tanks*

S ✹ *tanks*

VP ✹ *tanks*

P ✹ *with*

PP ✹ *with*

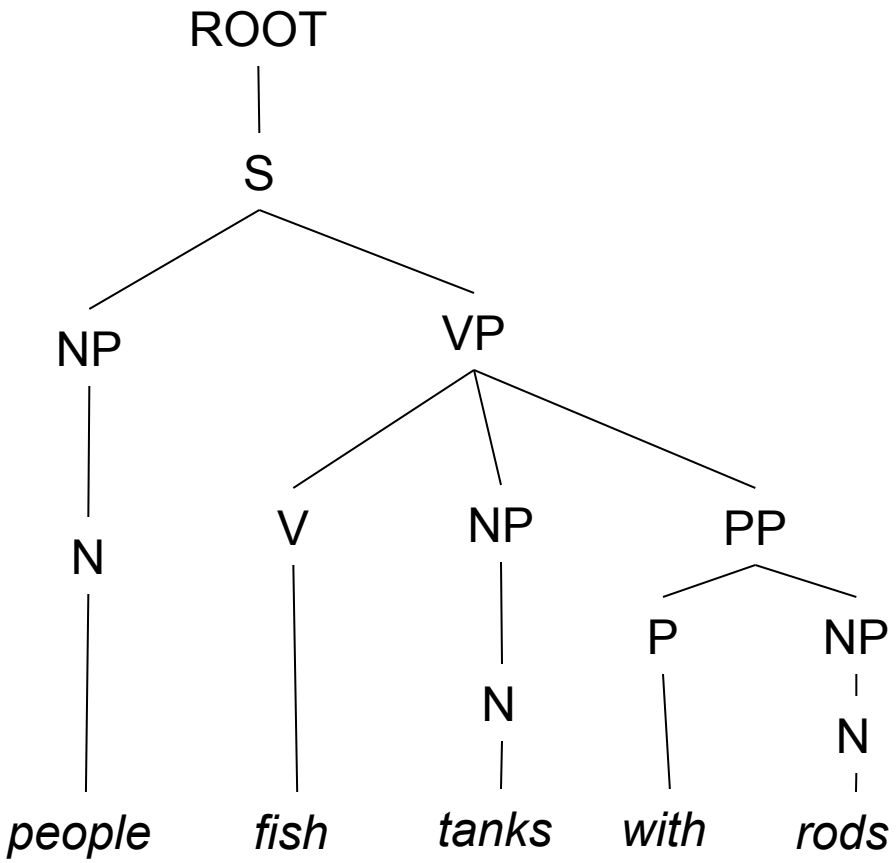
Chomsky Normal Form

- You should think of this as a transformation for efficient parsing
- With some extra book-keeping in symbol names, you can even reconstruct the same trees with a detransform
- In practice full Chomsky Normal Form is a pain
  - Reconstructing n-aries is easy
  - Reconstructing unaries/empties is trickier

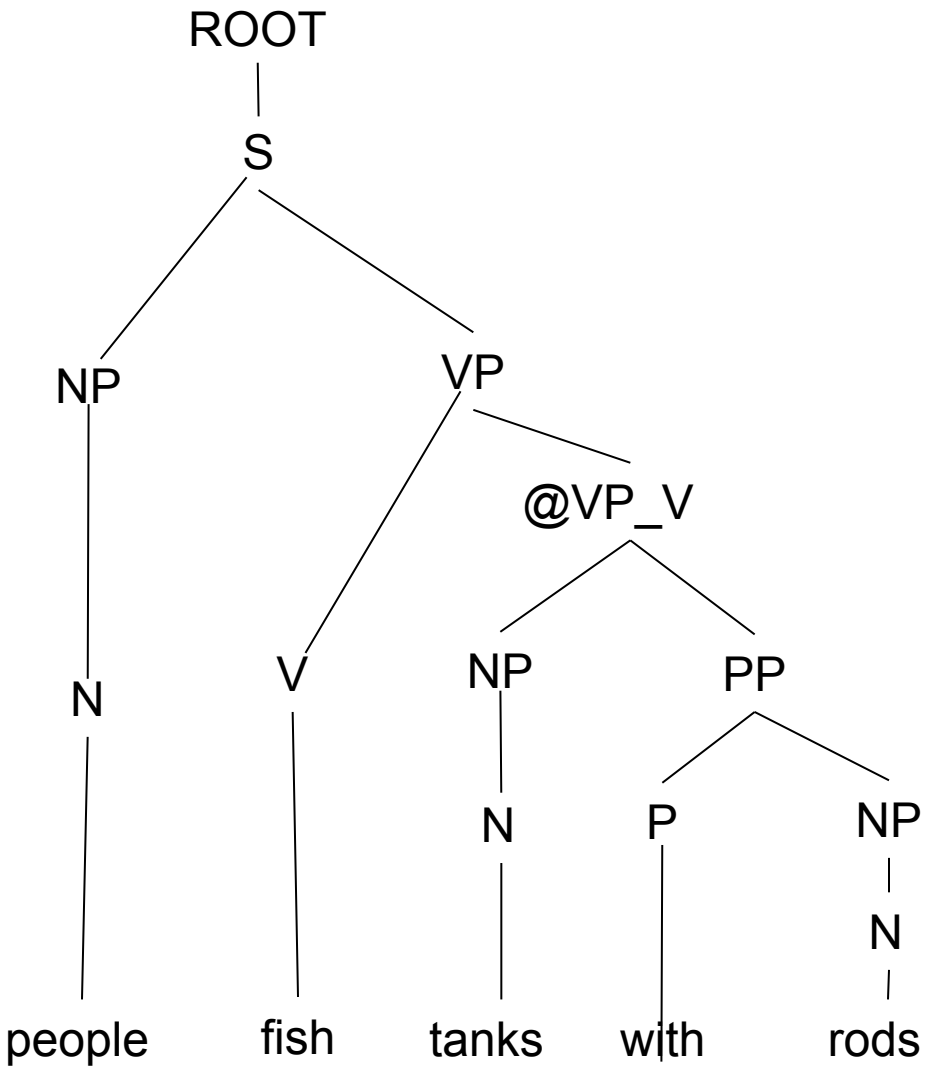
- **Binarization** is crucial for cubic time CFG parsing

- The rest isn't necessary; it just makes the algorithms cleaner and a bit quicker

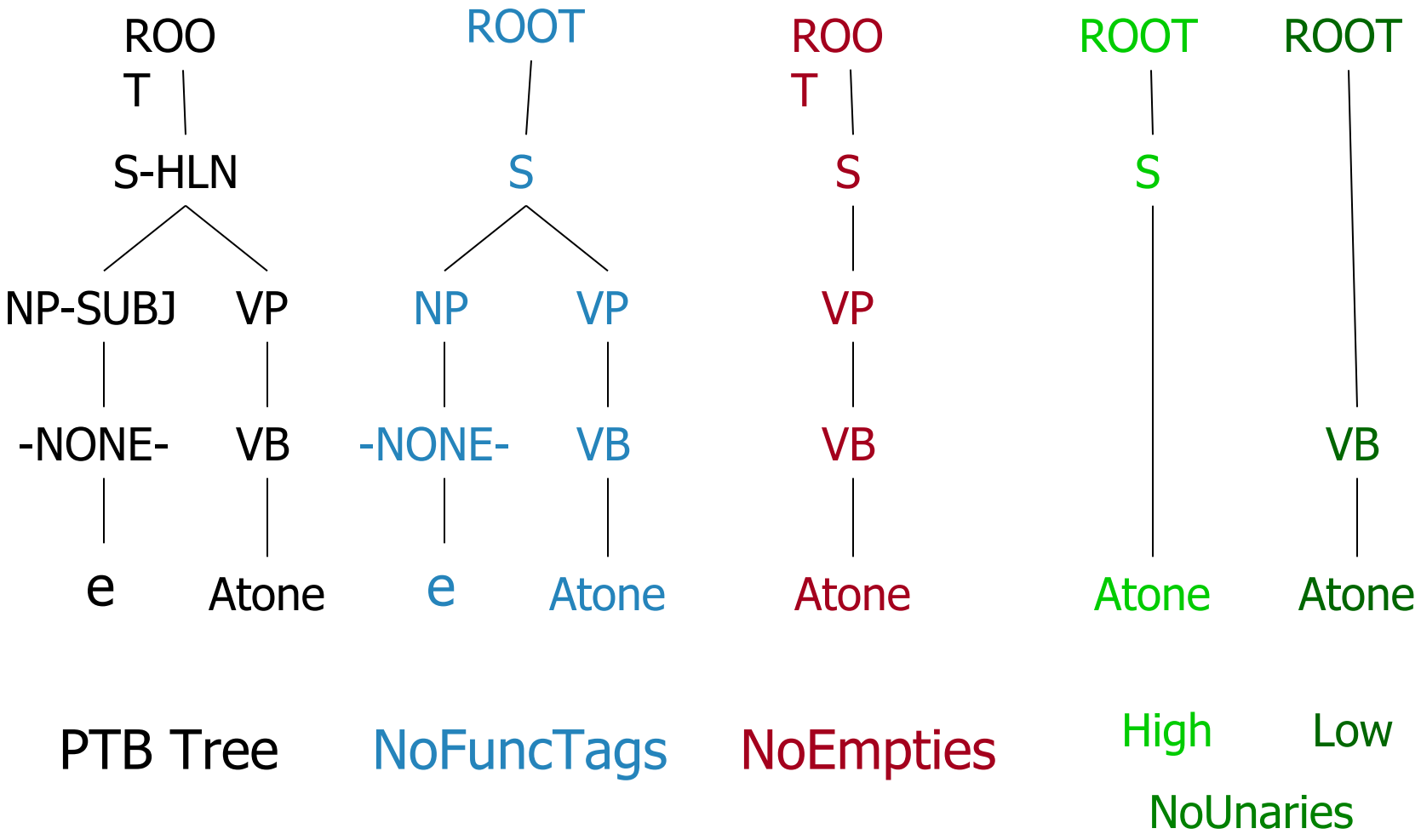
An example: before binarization…



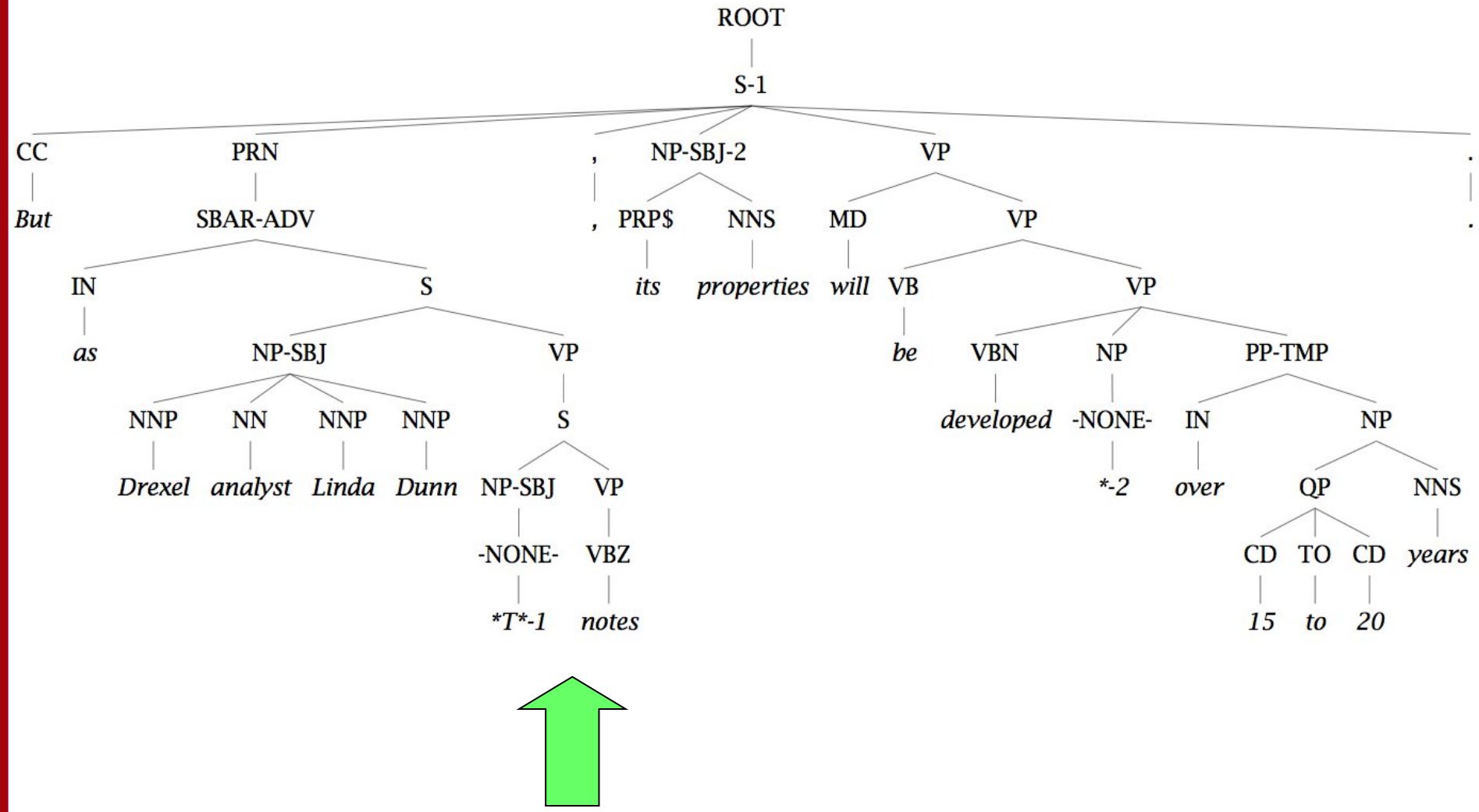
After binarization…



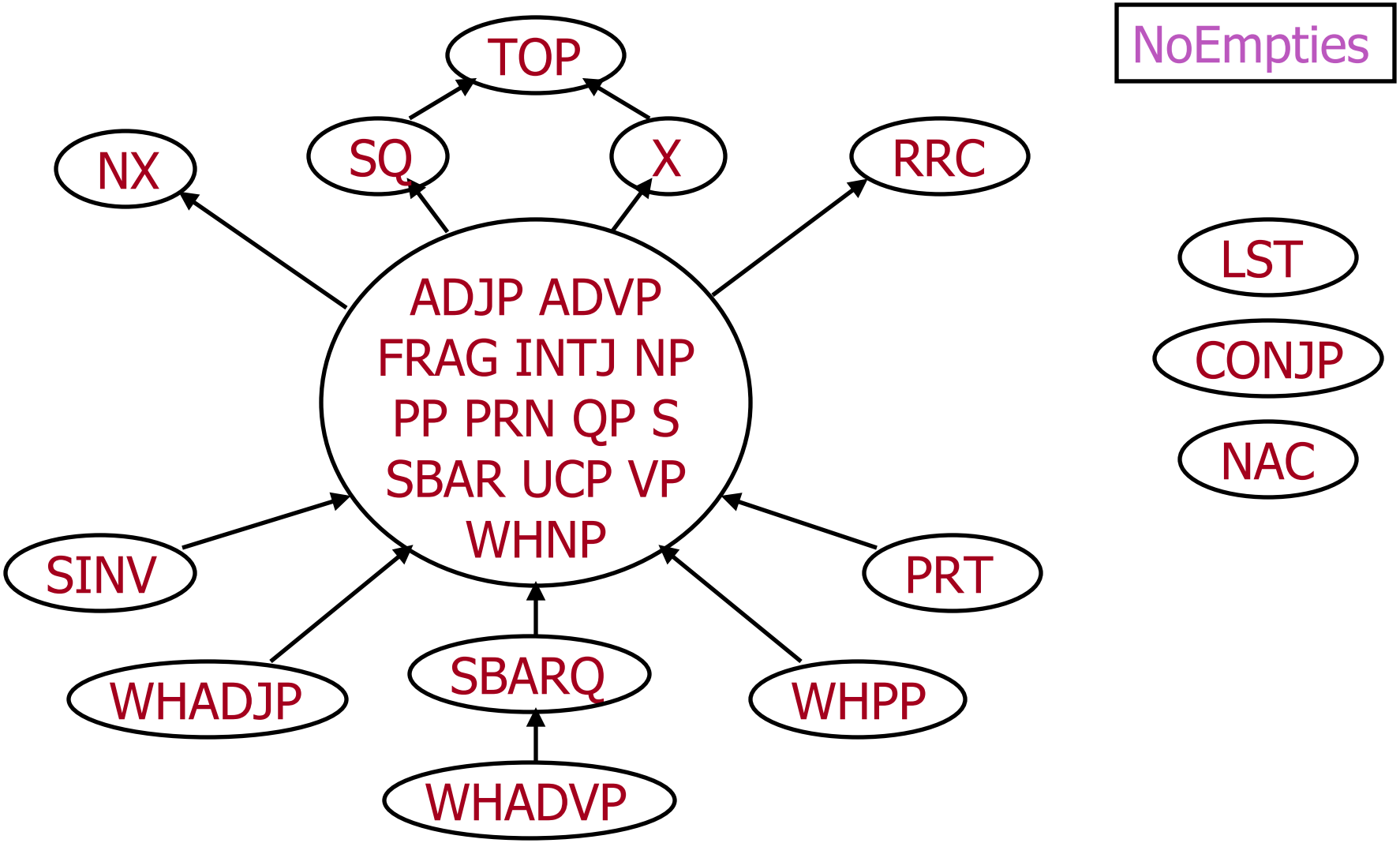
Treebank: empties and unaries



Unary rules: alchemy in the land of treebanks



Same-Span Reachability

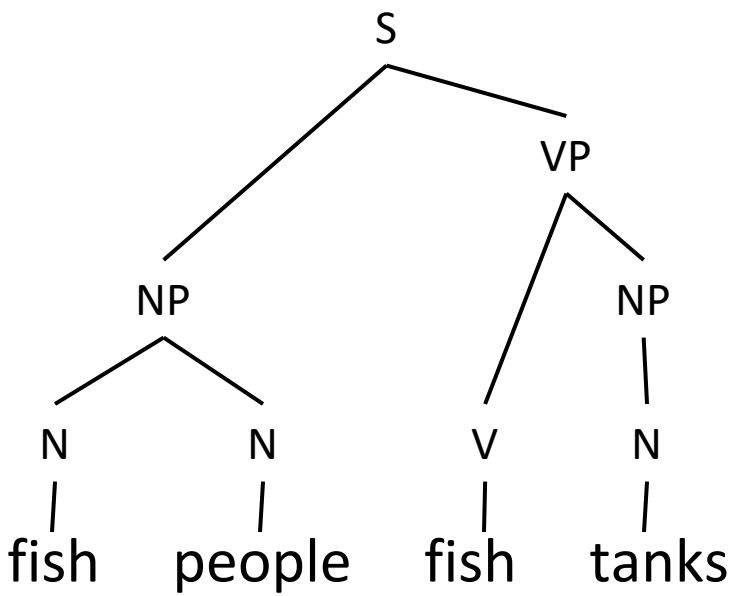




CKY Parsing

Exact polynomial time parsing of (P)CFGs

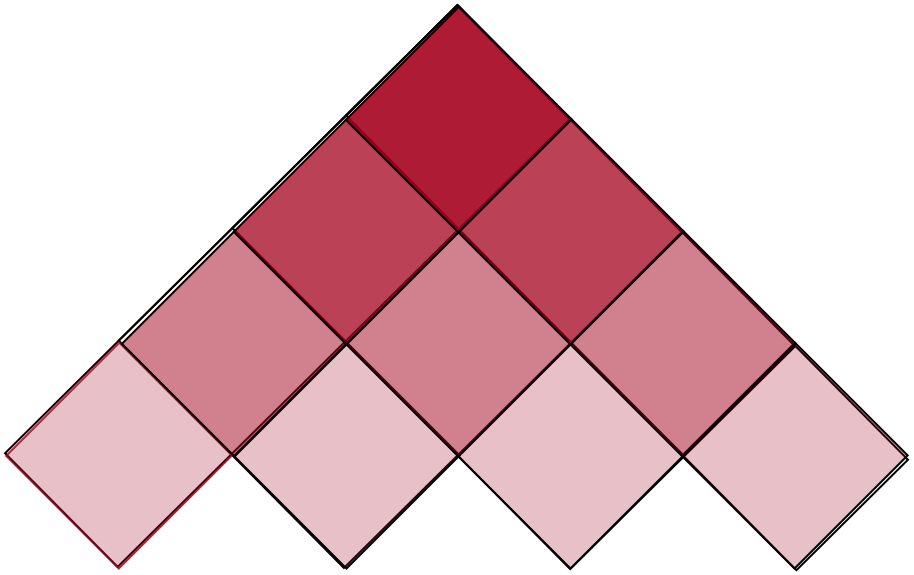
Constituency Parsing



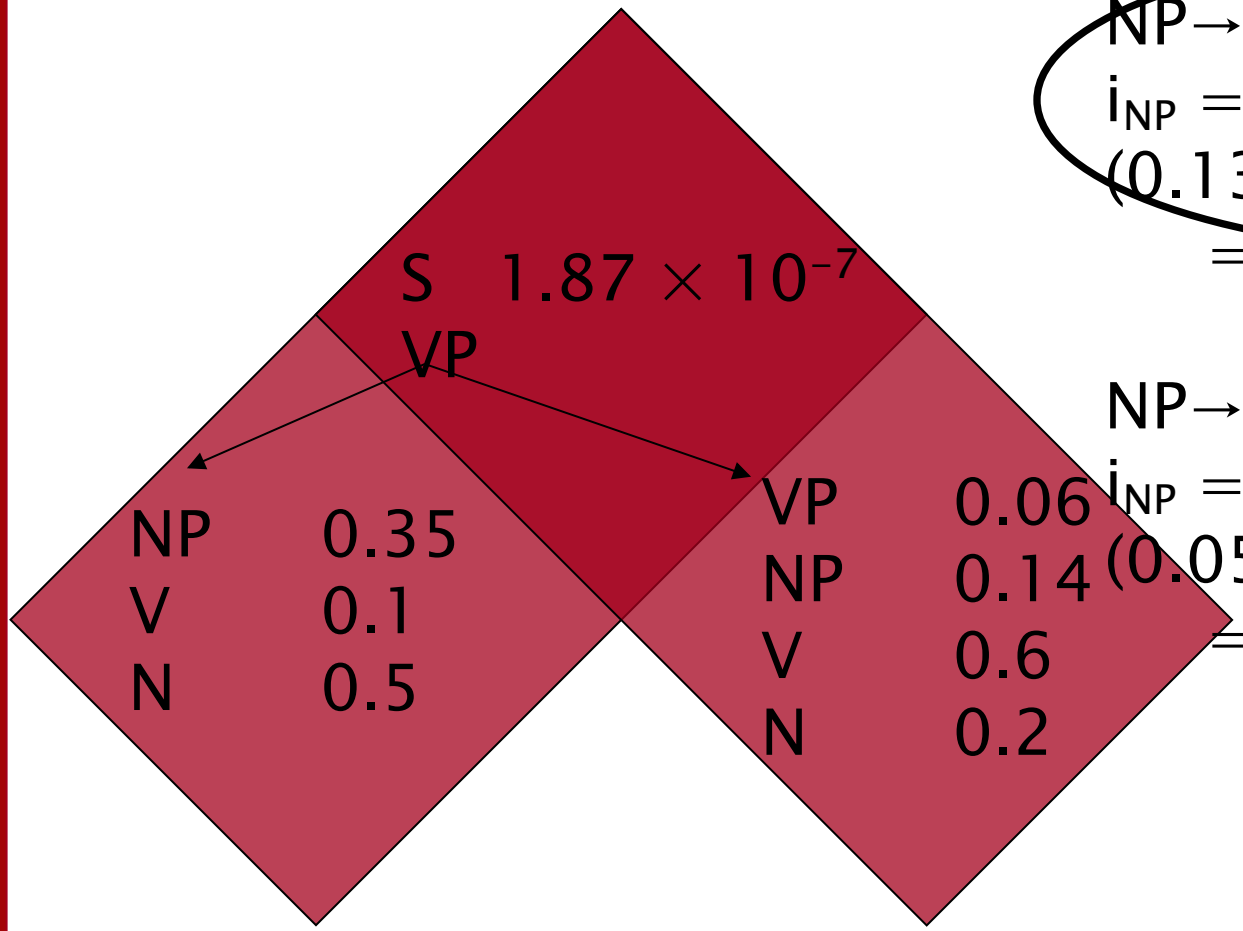
PCFG

Rule	Prob θ_i
S ✹ NP VP	θ_0
NP ✹ NP NP	θ_1
…	
N ✹ fish	θ_{42}
N ✹ people	θ_{43}
V ✹ fish	θ_{44}
…	

Cocke-Kasami-Younger (CKY) Constituency Parsing



Viterbi (Max) Scores



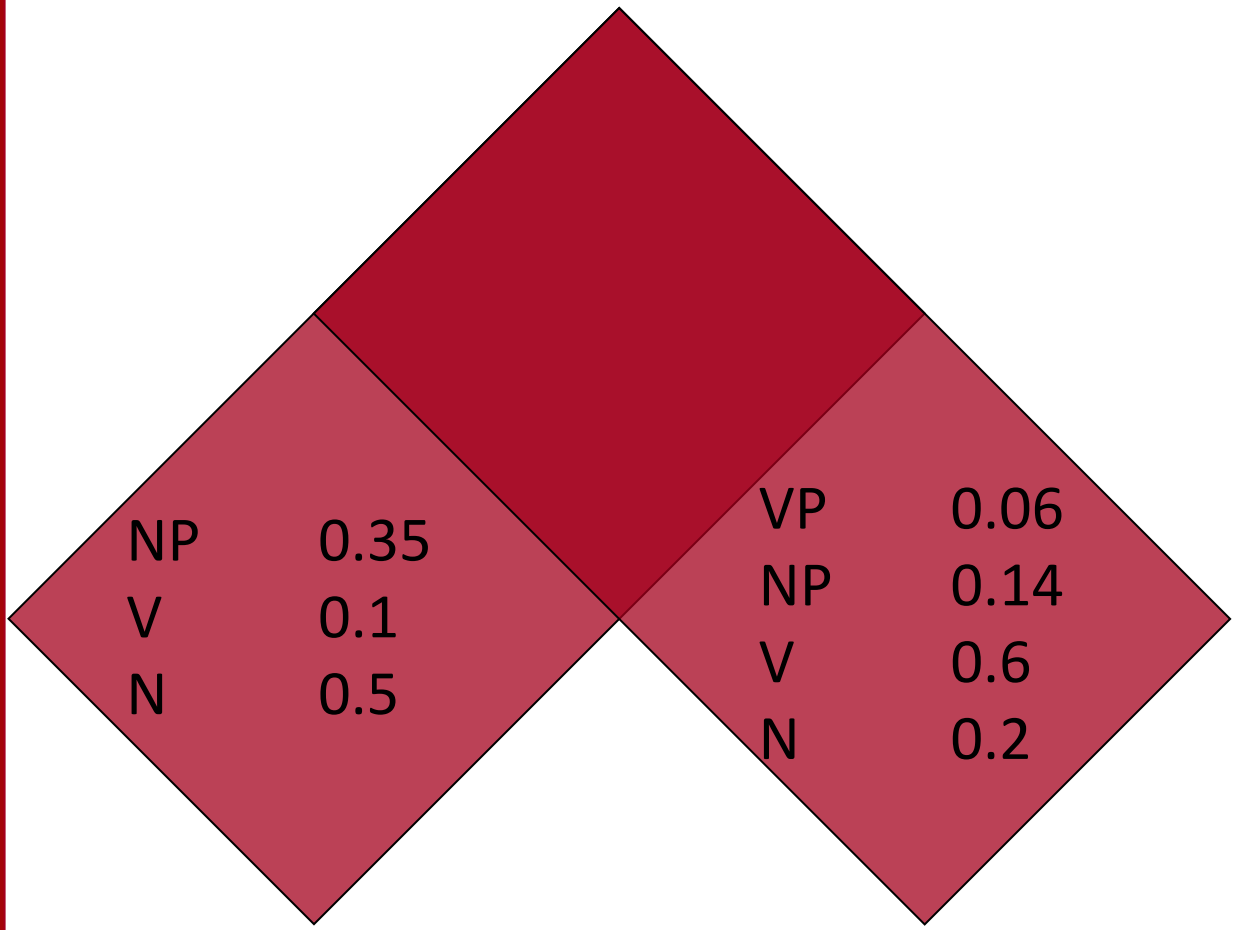
NP→NN NNS 0.13

i_{NP} = (0.13)(0.0023)(0.0014)
= 1.87×10^{-7}

NP→NNP NNS 0.056

i_{NP} = (0.056)(0.001)(0.0014)
= 7.84×10^{-8}

Viterbi (Max) Scores



S ✹ NP VP	0.9
S ✹ VP	0.1
VP ✹ V NP	0.5
VP ✹ V	0.1
VP ✹ V @VP_V	0.3
VP ✹ V PP	0.1
@VP_V ✹ NP PP	1.0
NP ✹ NP NP	0.1
NP ✹ NP PP	0.2
NP ✹ N	0.7
PP ✹ P NP	1.0

Extended CKY parsing

- Unaries can be incorporated into the algorithm
  - Messy, but doesn't increase algorithmic complexity
- Empties can be incorporated
  - Use fenceposts
  - Doesn't increase complexity; essentially like unaries

- Binarization is *vital*
  - Without binarization, you don't get parsing cubic in the length of the sentence and in the number of nonterminals in the grammar
    - Binarization may be an explicit transformation or implicit in how the parser works (Early-style dotted rules), but it's always there.

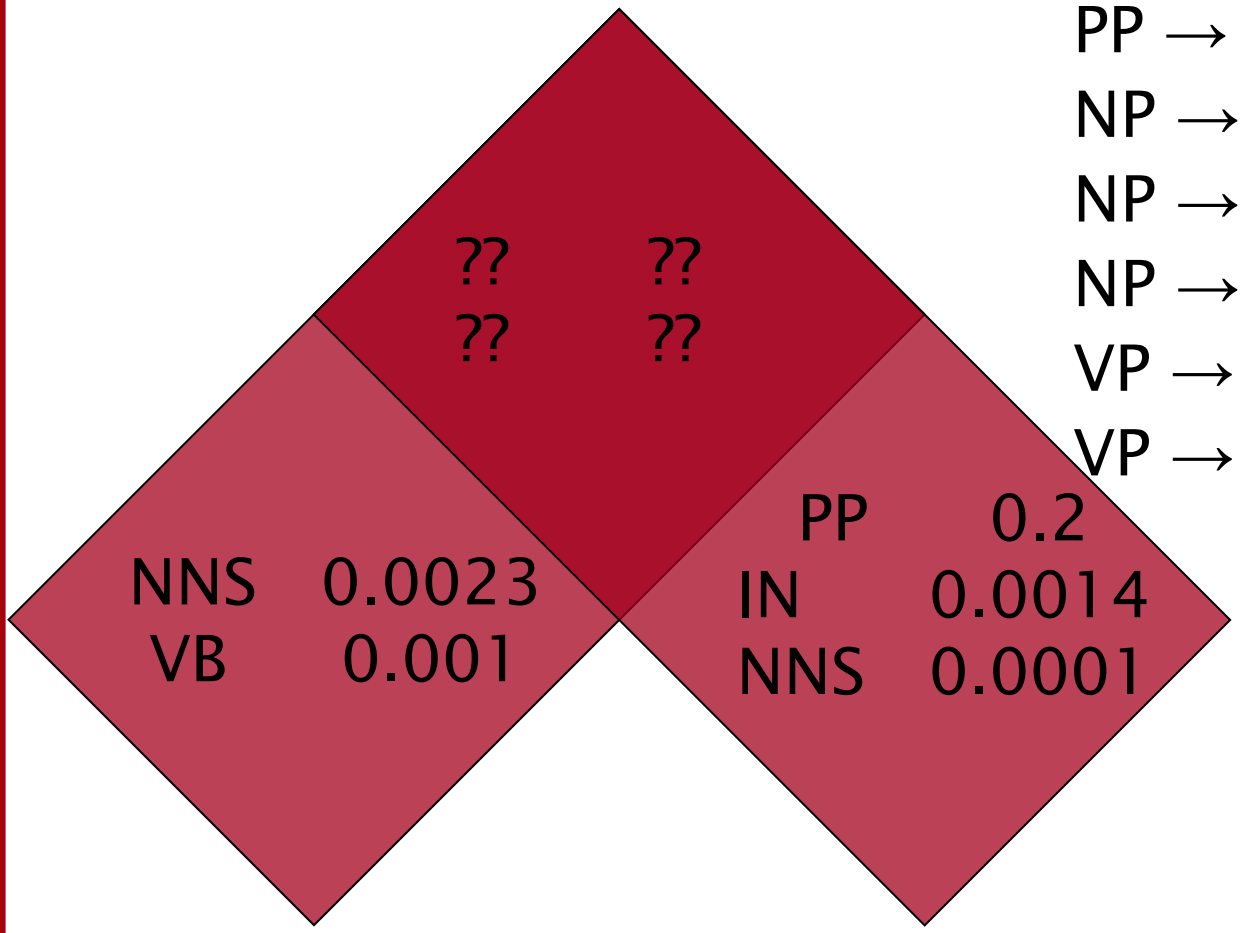
The CKY algorithm (1960/1965) … extended to unaries

```
function CKY(words, grammar) returns [most_probable_parse,prob]
  score = new double[#(words)+1][#(words)+1][#(nonterms)]
  back = new Pair[#(words)+1][#(words)+1][#nonterms]]
  for i=0; i<#(words); i++
    for A in nonterms
      if A -> words[i] in grammar
        score[i][i+1][A] = P(A -> words[i])
    //handle unaries
    boolean added = true
    while added
      added = false
      for A, B in nonterms
        if score[i][i+1][B] > 0 && A->B in grammar
          prob = P(A->B)*score[i][i+1][B]
          if prob > score[i][i+1][A]
            score[i][i+1][A] = prob
            back[i][i+1][A] = B
            added = true
```

The CKY algorithm (1960/1965) … extended to unaries

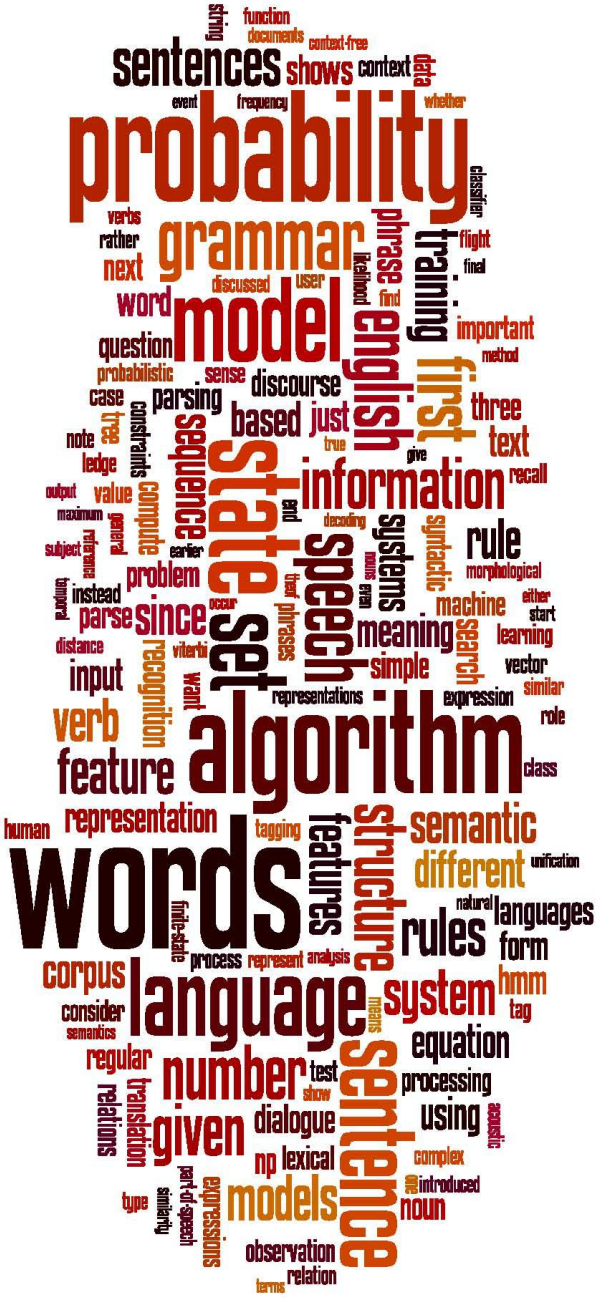
```
for span = 2 to #(words)
  for begin = 0 to #(words)- span
    end = begin + span
    for split = begin+1 to end-1
      for A,B,C in nonterms
        prob=score[begin][split][B]*score[split][end][C]*P(A->BC)
        if prob > score[begin][end][A]
          score[begin]end][A] = prob
          back[begin][end][A] = new Triple(split,B,C)
    //handle unaries
    boolean added = true
    while added
      added = false
      for A, B in nonterms
        prob = P(A->B)*score[begin][end][B];
        if prob > score[begin][end][A]
          score[begin][end][A] = prob
          back[begin][end][A] = B
          added = true
return buildTree(score, back)
```

Quiz Question!



PP → IN	0.002
NP → NNS NNS	0.01
NP → NNS NP	0.005
NP → NNS PP	0.01
VP → VB PP	0.045
VP → VB NP	0.015

What constituents (with what probability can you make?



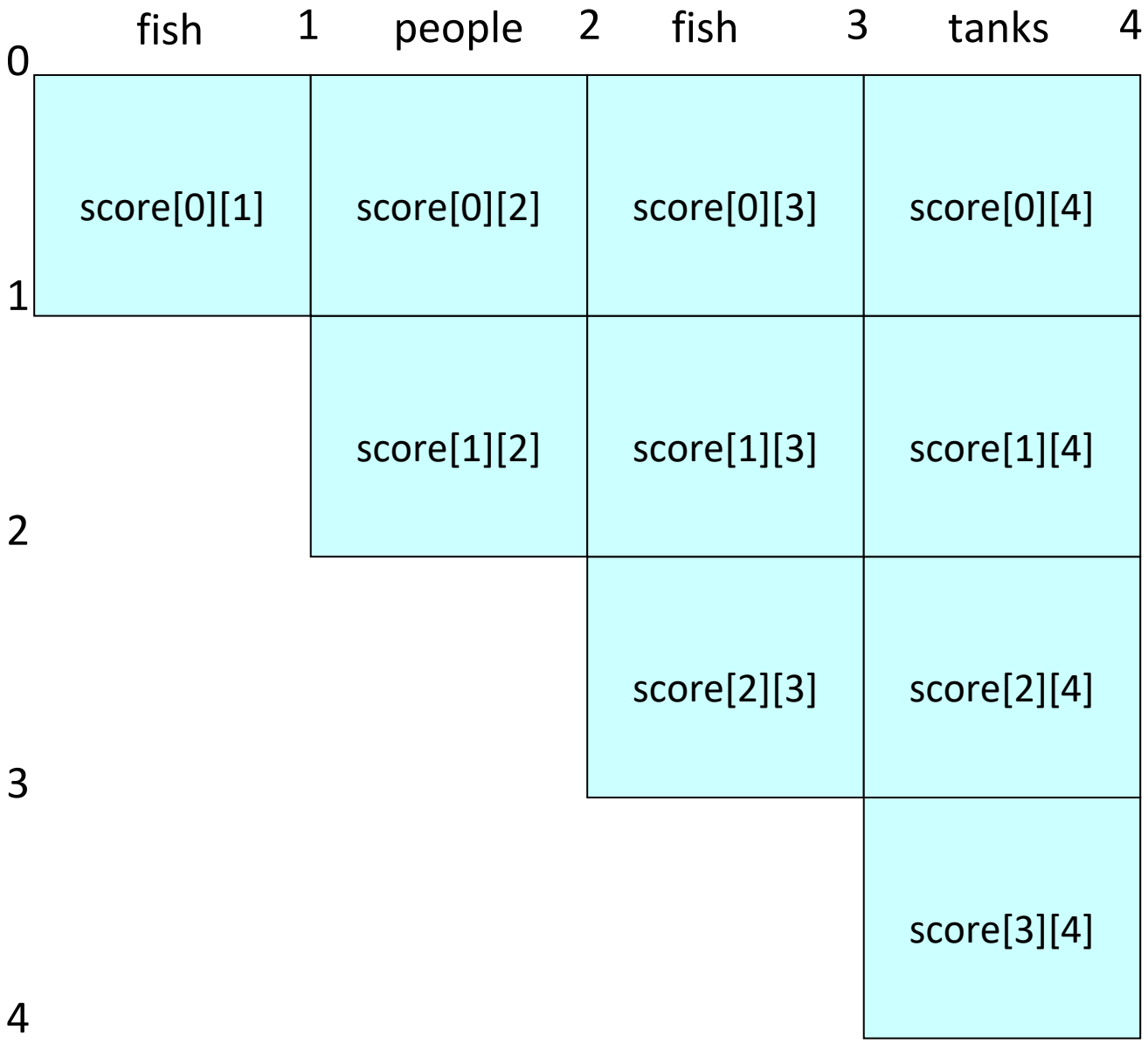
CKY Parsing

A worked example

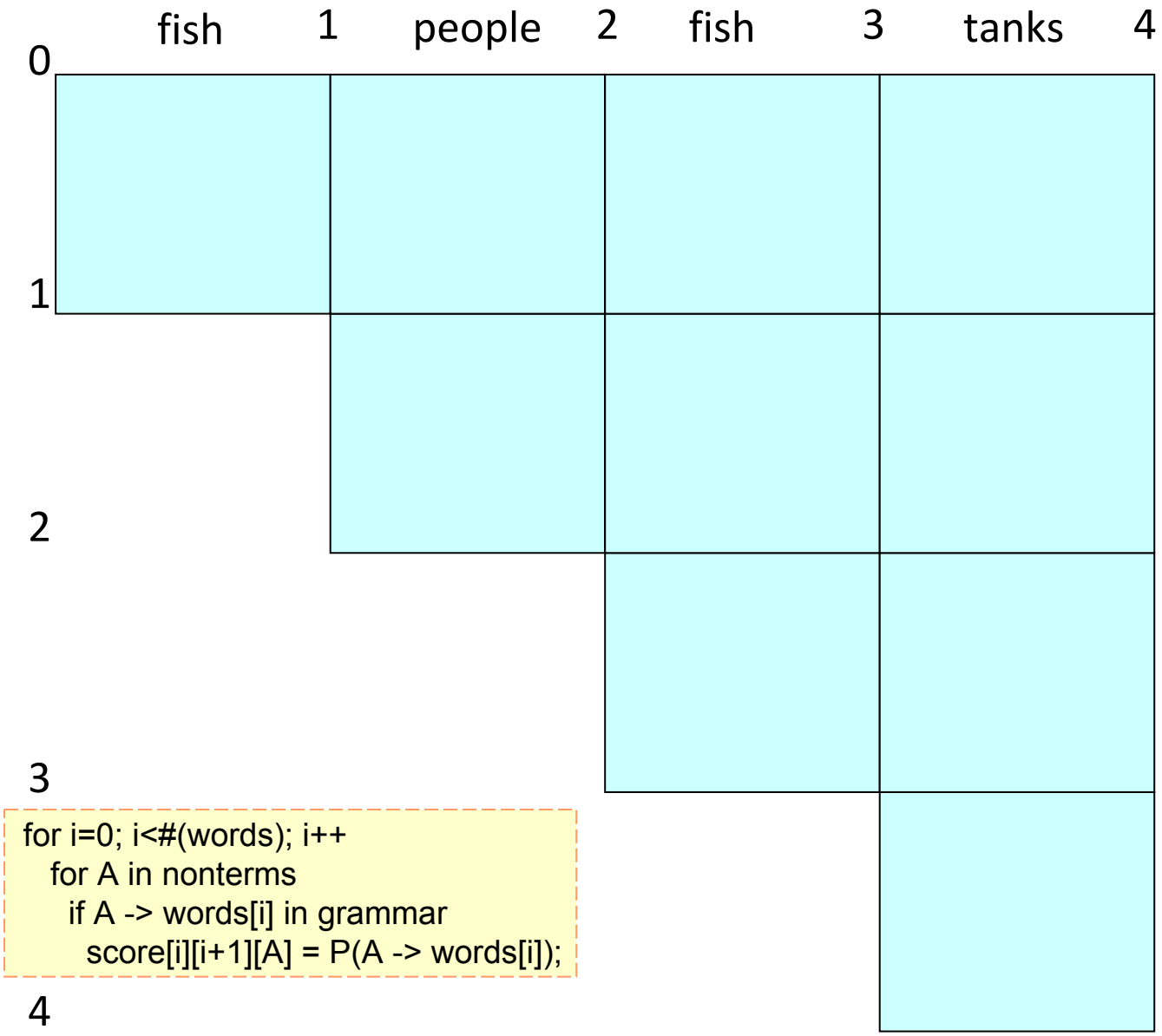
The grammar:
Binary, no epsilons,

S ✹ NP VP	0.9
S ✹ VP	0.1
VP ✹ V NP	0.5
VP ✹ V	0.1
VP ✹ V @VP_V	0.3
VP ✹ V PP	0.1
@VP_V ✹ NP PP	1.0
NP ✹ NP NP	0.1
NP ✹ NP PP	0.2
NP ✹ N	0.7
PP ✹ P NP	1.0

N ✹ *people*	0.5
N ✹ *fish*	0.2
N ✹ *tanks*	0.2
N ✹ *rods*	0.1
V ✹ *people*	0.1
V ✹ *fish*	0.6
V ✹ *tanks*	0.3
P ✹ *with*	1.0


fish
1
people
2
fish
3
tanks
4
0
1
2
3
4
score[0][1]
score[0][2]
score[0][3]
score[0][4]
score[1][2]
score[1][3]
score[1][4]
score[2][3]
score[2][4]
score[3][4]

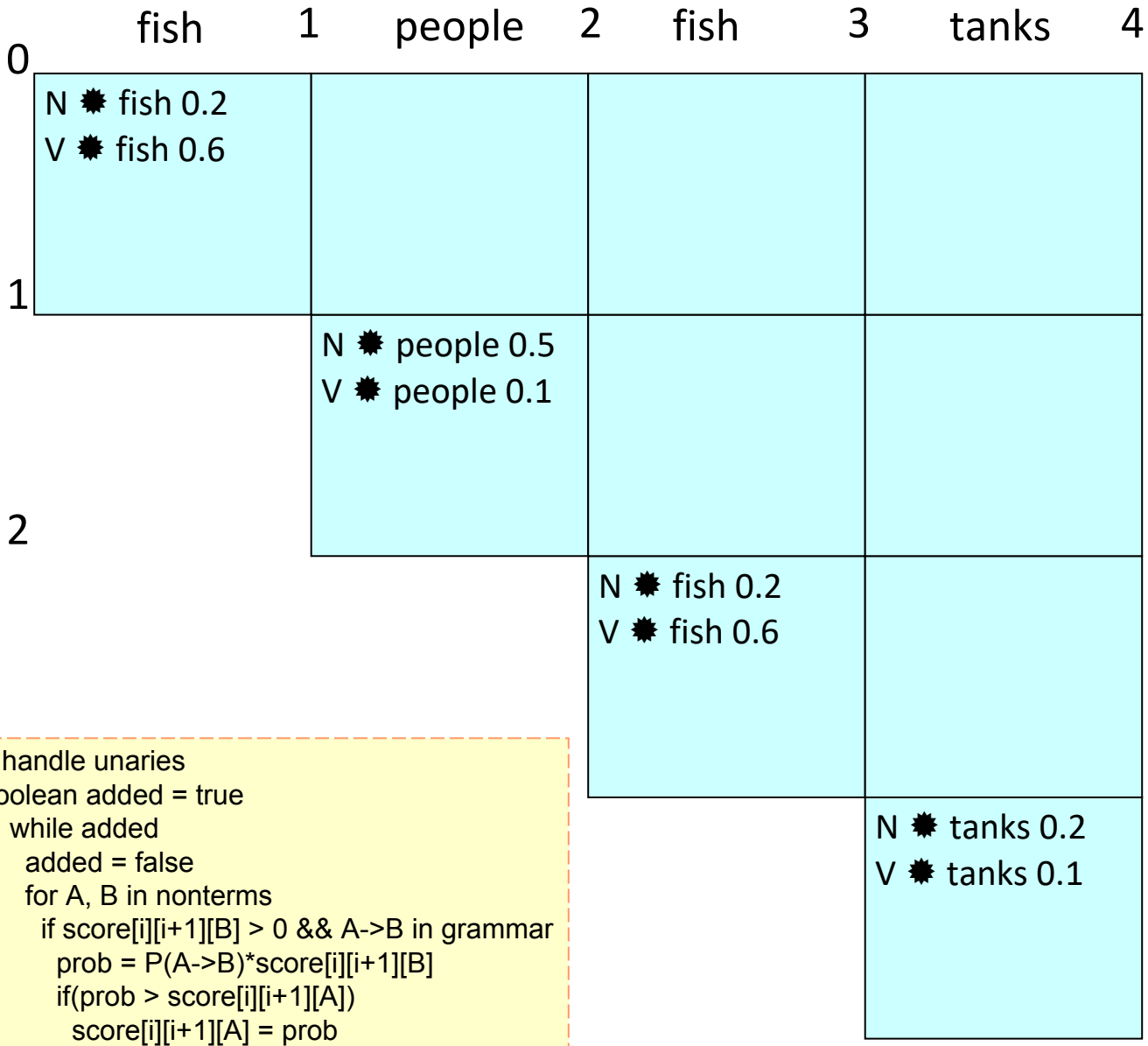
S ✹ NP VP	0.9
S ✹ VP	0.1
VP ✹ V NP	0.5
VP ✹ V	0.1
VP ✹ V @VP_V	0.3
VP ✹ V PP	0.1
@VP_V ✹ NP PP	1.0
NP ✹ NP NP	0.1
NP ✹ NP PP	0.2
NP ✹ N	0.7
PP ✹ P NP	1.0
N ✹ *people*	0.5
N ✹ *fish*	0.2
N ✹ *tanks*	0.2
N ✹ *rods*	0.1
V ✹ *people*	0.1
V ✹ *fish*	0.6
V ✹ *tanks*	0.3
P ✹ *with*	1.0




```
for i=0; i<#(words); i++
   for A in nonterms
      if A -> words[i] in grammar
         score[i][i+1][A] = P(A -> words[i]);
```

S ✹ NP VP	0.9
S ✹ VP	0.1
VP ✹ V NP	0.5
VP ✹ V	0.1
VP ✹ V @VP_V	0.3
VP ✹ V PP	0.1
@VP_V ✹ NP PP	1.0
NP ✹ NP NP	0.1
NP ✹ NP PP	0.2
NP ✹ N	0.7
PP ✹ P NP	1.0

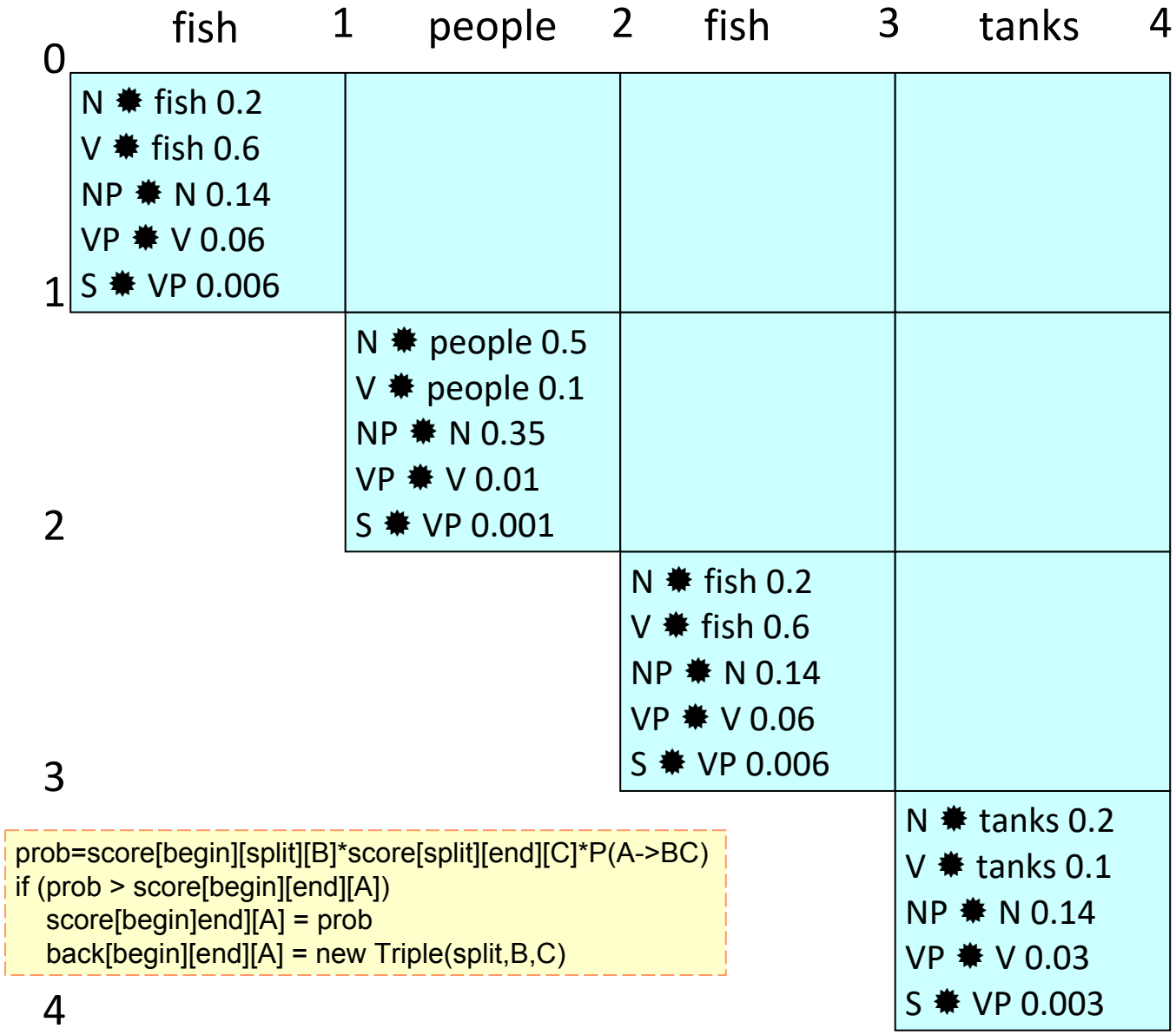
N ✹ *people*	0.5
N ✹ *fish*	0.2
N ✹ *tanks*	0.2
N ✹ *rods*	0.1
V ✹ *people*	0.1
V ✹ *fish*	0.6
V ✹ *tanks*	0.3
P ✹ *with*	1.0




```
// handle unaries
boolean added = true
    while added
      added = false
      for A, B in nonterms
        if score[i][i+1][B] > 0 && A->B in grammar
          prob = P(A->B)*score[i][i+1][B]
          if(prob > score[i][i+1][A])
            score[i][i+1][A] = prob
            back[i][i+1][A] = B
            added = true
```

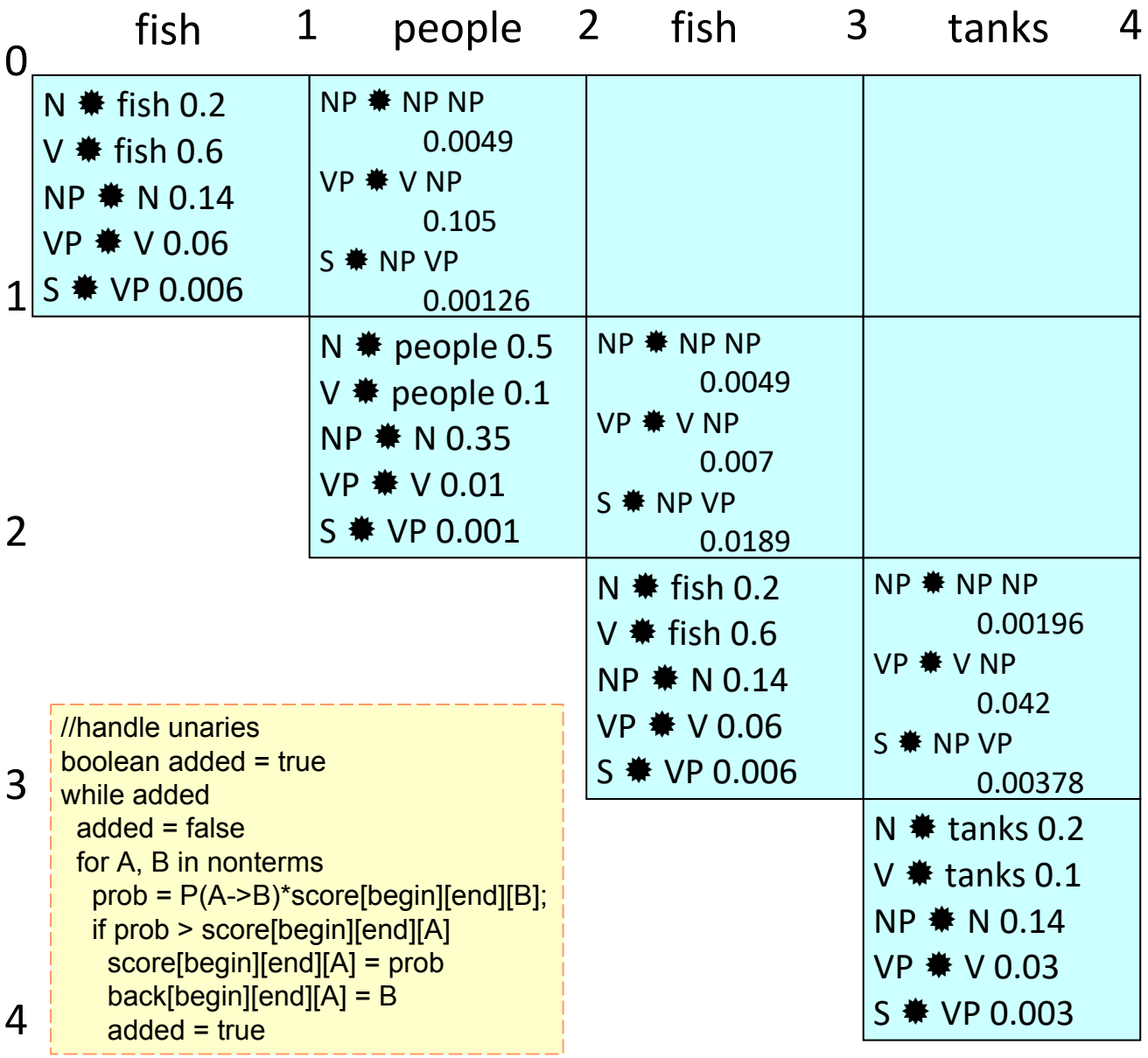
S ✹ NP VP	0.9
S ✹ VP	0.1
VP ✹ V NP	0.5
VP ✹ V	0.1
VP ✹ V @VP_V	0.3
VP ✹ V PP	0.1
@VP_V ✹ NP PP	1.0
NP ✹ NP NP	0.1
NP ✹ NP PP	0.2
NP ✹ N	0.7
PP ✹ P NP	1.0

N ✹ *people*	0.5
N ✹ *fish*	0.2
N ✹ *tanks*	0.2
N ✹ *rods*	0.1
V ✹ *people*	0.1
V ✹ *fish*	0.6
V ✹ *tanks*	0.3
P ✹ *with*	1.0



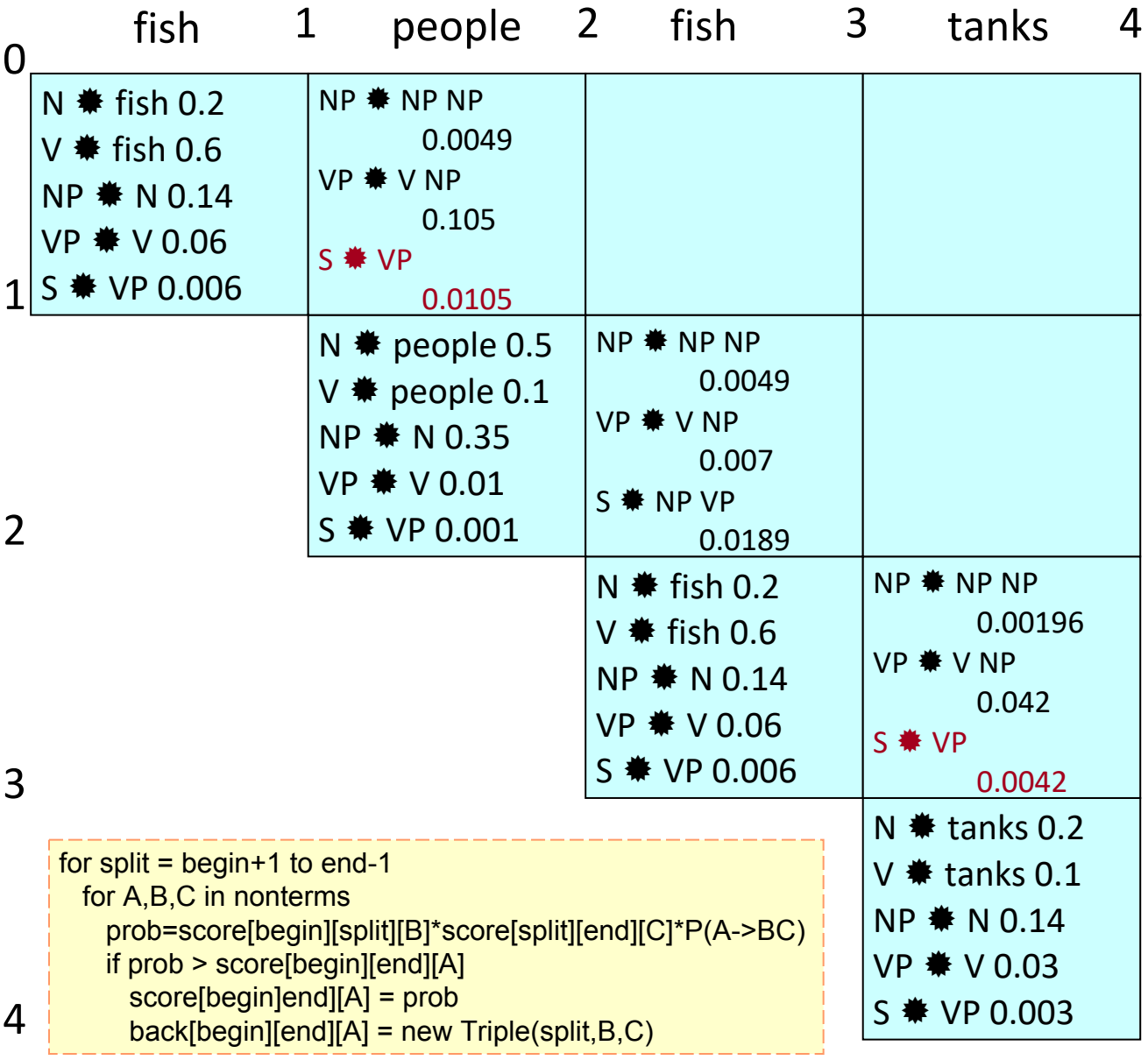
S ✹ NP VP	0.9
S ✹ VP	0.1
VP ✹ V NP	0.5
VP ✹ V	0.1
VP ✹ V @VP_V	0.3
VP ✹ V PP	0.1
@VP_V ✹ NP PP	1.0
NP ✹ NP NP	0.1
NP ✹ NP PP	0.2
NP ✹ N	0.7
PP ✹ P NP	1.0

N ✹ *people*	0.5
N ✹ *fish*	0.2
N ✹ *tanks*	0.2
N ✹ *rods*	0.1
V ✹ *people*	0.1
V ✹ *fish*	0.6
V ✹ *tanks*	0.3
P ✹ *with*	1.0



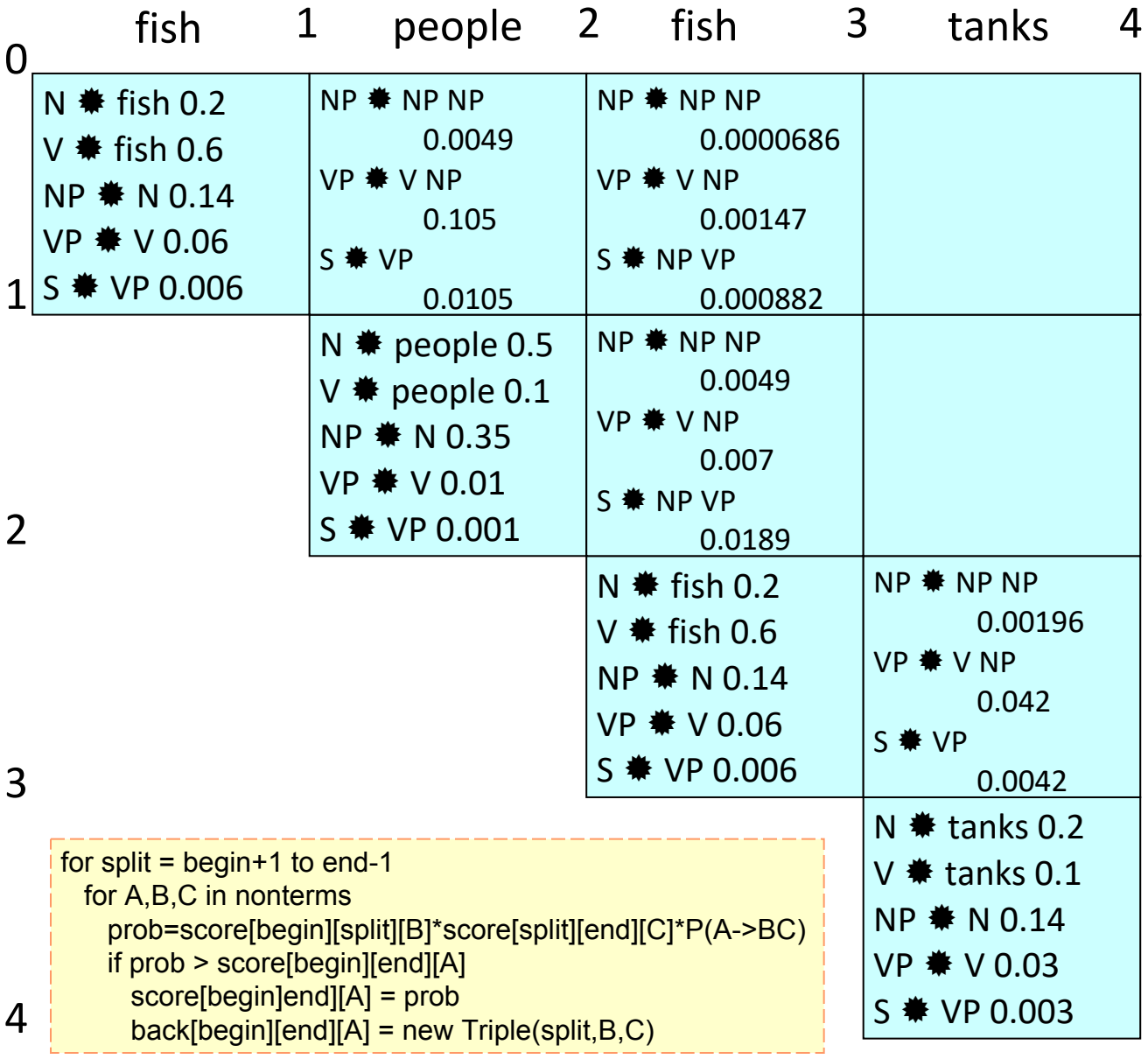
S ✹ NP VP	0.9
S ✹ VP	0.1
VP ✹ V NP	0.5
VP ✹ V	0.1
VP ✹ V @VP_V	0.3
VP ✹ V PP	0.1
@VP_V ✹ NP PP	1.0
NP ✹ NP NP	0.1
NP ✹ NP PP	0.2
NP ✹ N	0.7
PP ✹ P NP	1.0

N ✹ *people*	0.5
N ✹ *fish*	0.2
N ✹ *tanks*	0.2
N ✹ *rods*	0.1
V ✹ *people*	0.1
V ✹ *fish*	0.6
V ✹ *tanks*	0.3
P ✹ *with*	1.0



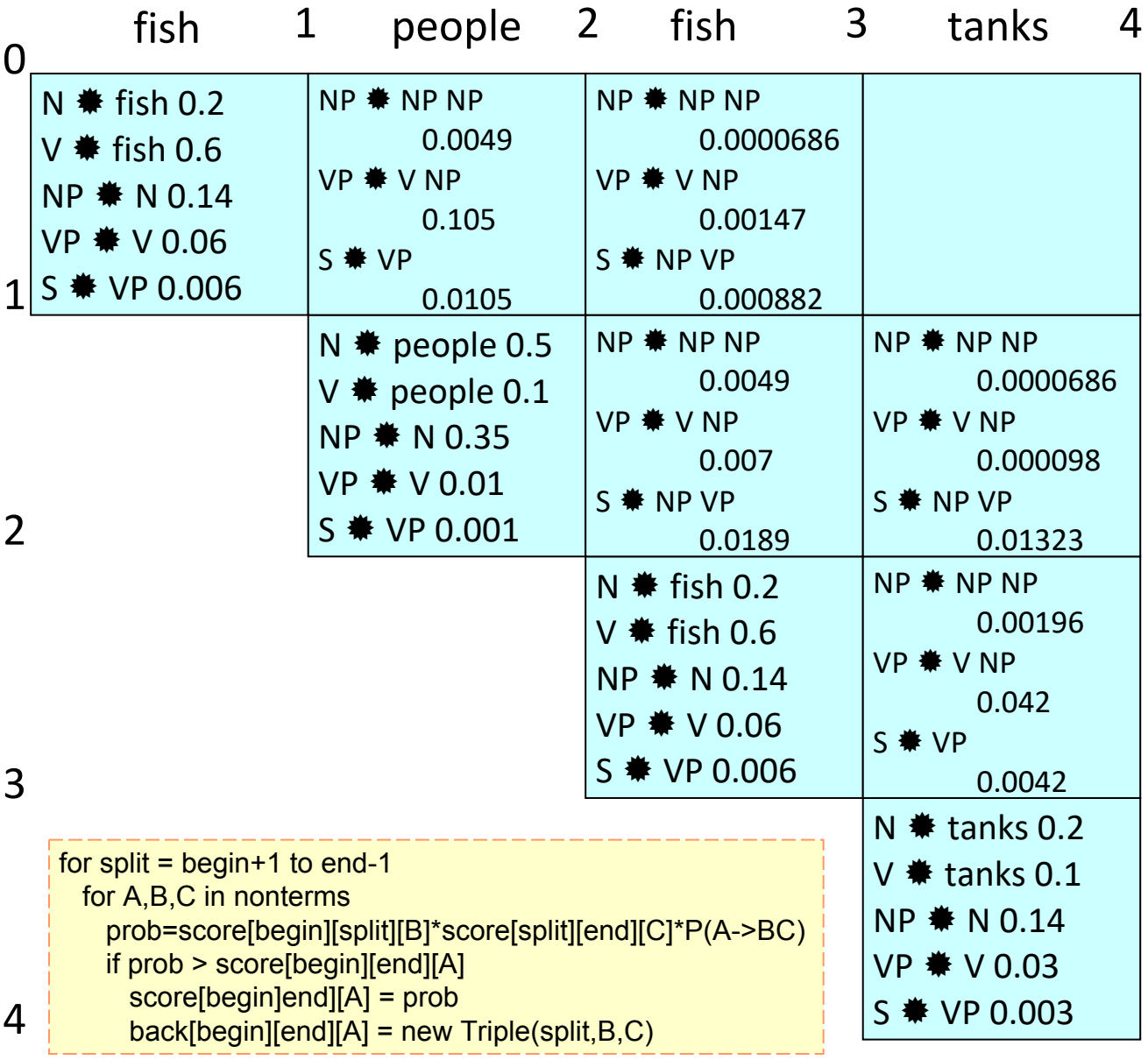
S ✹ NP VP	0.9
S ✹ VP	0.1
VP ✹ V NP	0.5
VP ✹ V	0.1
VP ✹ V @VP_V	0.3
VP ✹ V PP	0.1
@VP_V ✹ NP PP	1.0
NP ✹ NP NP	0.1
NP ✹ NP PP	0.2
NP ✹ N	0.7
PP ✹ P NP	1.0

N ✹ *people*	0.5
N ✹ *fish*	0.2
N ✹ *tanks*	0.2
N ✹ *rods*	0.1
V ✹ *people*	0.1
V ✹ *fish*	0.6
V ✹ *tanks*	0.3
P ✹ *with*	1.0



S ✹ NP VP	0.9
S ✹ VP	0.1
VP ✹ V NP	0.5
VP ✹ V	0.1
VP ✹ V @VP_V	0.3
VP ✹ V PP	0.1
@VP_V ✹ NP PP	1.0
NP ✹ NP NP	0.1
NP ✹ NP PP	0.2
NP ✹ N	0.7
PP ✹ P NP	1.0

N ✹ *people*	0.5
N ✹ *fish*	0.2
N ✹ *tanks*	0.2
N ✹ *rods*	0.1
V ✹ *people*	0.1
V ✹ *fish*	0.6
V ✹ *tanks*	0.3
P ✹ *with*	1.0



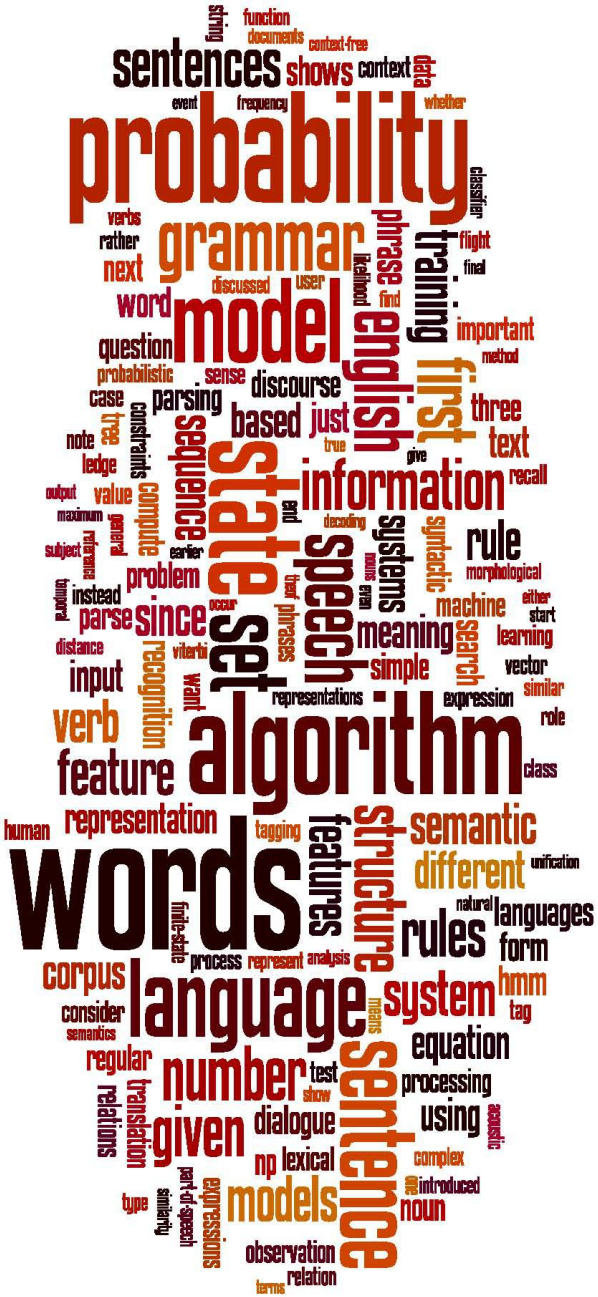

```
for split = begin+1 to end-1
  for A,B,C in nonterms
    prob=score[begin][split][B]*score[split][end][C]*P(A->BC)
    if prob > score[begin][end][A]
      score[begin]end][A] = prob
      back[begin][end][A] = new Triple(split,B,C)
```

S ✹ NP VP	0.9
S ✹ VP	0.1
VP ✹ V NP	0.5
VP ✹ V	0.1
VP ✹ V @VP_V	0.3
VP ✹ V PP	0.1
@VP_V ✹ NP PP	1.0
NP ✹ NP NP	0.1
NP ✹ NP PP	0.2
NP ✹ N	0.7
PP ✹ P NP	1.0

N ✹ *people*	0.5
N ✹ *fish*	0.2
N ✹ *tanks*	0.2
N ✹ *rods*	0.1
V ✹ *people*	0.1
V ✹ *fish*	0.6
V ✹ *tanks*	0.3
P ✹ *with*	1.0

	fish	1	people	2	fish	3	tanks	4
0	N ✹ fish 0.2 V ✹ fish 0.6 NP ✹ N 0.14 VP ✹ V 0.06 S ✹ VP 0.006		NP ✹ NP NP 0.0049 VP ✹ V NP 0.105 S ✹ VP 0.0105		NP ✹ NP NP 0.0000686 VP ✹ V NP 0.00147 S ✹ NP VP 0.000882		NP ✹ NP NP 0.0000009604 VP ✹ V NP 0.00002058 S ✹ NP VP 0.0018522	
1			N ✹ people 0.5 V ✹ people 0.1 NP ✹ N 0.35 VP ✹ V 0.01 S ✹ VP 0.001		NP ✹ NP NP 0.0049 VP ✹ V NP 0.007 S ✹ NP VP 0.0189		NP ✹ NP NP 0.0000686 VP ✹ V NP 0.000098 S ✹ NP VP 0.01323	
2					N ✹ fish 0.2 V ✹ fish 0.6 NP ✹ N 0.14 VP ✹ V 0.06 S ✹ VP 0.006		NP ✹ NP NP 0.00196 VP ✹ V NP 0.042 S ✹ VP 0.0042	
3							N ✹ tanks 0.2 V ✹ tanks 0.1 NP ✹ N 0.14 VP ✹ V 0.03 S ✹ VP 0.003	
4								

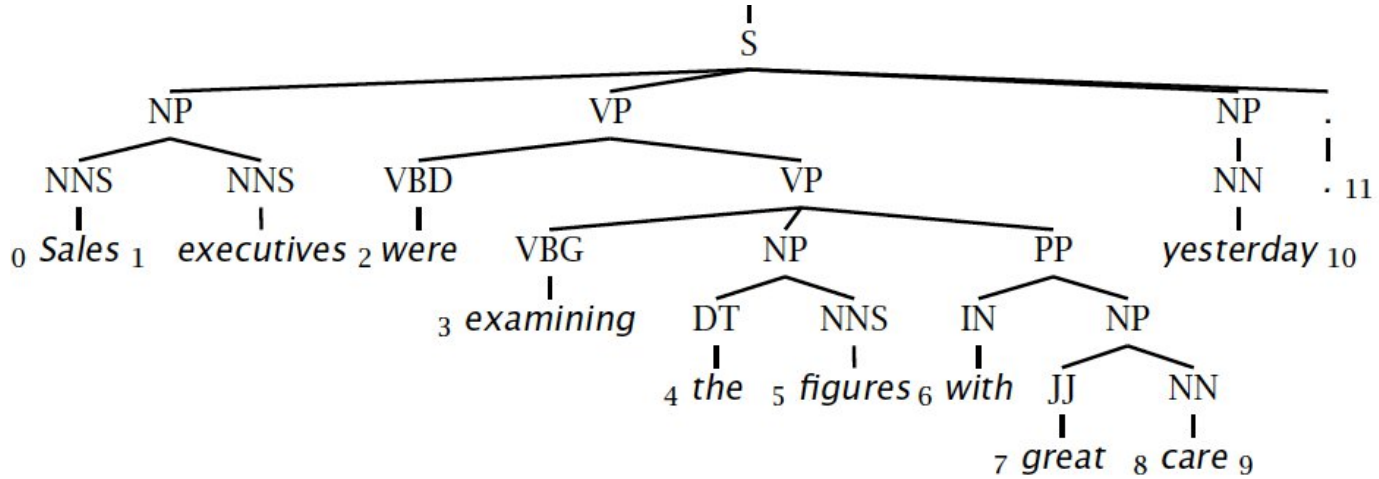
Call buildTree(score, back) to get the best parse



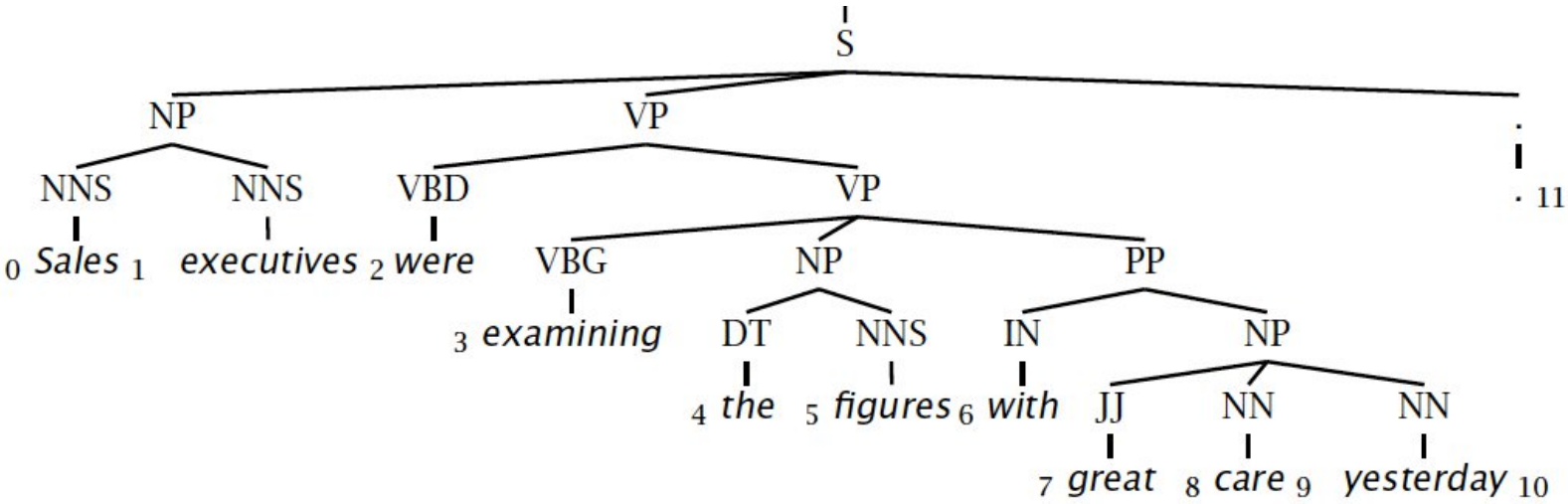
Constituency Parser Evaluation

Evaluating constituency parsing

Gold standard brackets: **S-(0:11)**, **NP-(0:2)**, VP-(2:9), VP-(3:9), **NP-(4:6)**, PP-(6-9), NP-(7,9), NP-(9:10)



Candidate brackets: **S-(0:11)**, **NP-(0:2)**, VP-(2:10), VP-(3:10), **NP-(4:6)**, PP-(6-10), NP-(7,10)



Evaluating constituency parsing

Gold standard brackets:

S-(0:11), NP-(0:2), VP-(2:9), VP-(3:9), **NP-(4:6)**, PP-(6-9), NP-(7,9), NP-(9:10)

Candidate brackets:

S-(0:11), **NP-(0:2)**, VP-(2:10), VP-(3:10), **NP-(4:6)**, PP-(6-10), NP-(7,10)

Labeled Precision	3/7 = 42.9%
Labeled Recall	3/8 = 37.5%
LP/LR F1	40.0%
Tagging Accuracy	11/11 = 100.0%

How good are PCFGs?

- Penn WSJ parsing accuracy: about 73% LP/LR F1
- Robust
  - Usually admit everything, but with low probability
- Partial solution for grammar ambiguity
  - A PCFG gives some idea of the plausibility of a parse
  - But not so good because the independence assumptions are too strong
- Give a probabilistic language model
  - But in the simple case it performs worse than a trigram model
- The problem seems to be that PCFGs lack the lexicalization of a trigram model